11 Building with Gradle and Maven

This chapter covers

- Why build tools matter for a well-grounded developer
- Maven
- Gradle

The JDK ships with a compiler to turn Java source code into class files, as we saw in chapter 4. Despite that fact, few projects of any size rely just on `javac`. Let's start by looking at why a well-grounded developer should invest in familiarity with this layer of tooling.

11.1 Why build tools matter for a well-grounded developer

Build tools are the norm for the following reasons:

- Automating tedious operations
- Managing dependencies
- Ensuring consistency between developers

Although many options exist, two choices dominate the landscape today: Maven and Gradle. Understanding what these tools aim to solve, digging below the surface of how they get their job done, and understanding the differences between them—and how to extend them—will pay off for the well-grounded developer.

.1.1 Automating tedious operations

`javac` can turn any Java source file into a class file, but there's more to building a typical Java project than that. Just getting all the files properly listed to the compiler could be tedious in a large project if done by hand. Build tools provide defaults for finding code and let you easily configure if you have a nonstandard layout instead.

The conventional layout popularized by Maven, and used by default by Gradle as well, looks like this:

```
.
└── src
    ├── main                    (1)
    │   └── java                (2)
    │       └── com             (3)
    │           └── wellgrounded
    │               └── Main.java
    └── test
        └── java
            └── com
                └── wellgrounded
                    └── MainTest.java
```

(1) main and test separate our production code from our test code.

(2) Multiple languages easily coexist within one project with this structure.

(3) Further directory structure typically mirrors your package hierarchy.

As you can see, testing is baked all the way into the layout of our code. Java's come a long way since the time when folks used to ask whether they really needed to write tests for their code. The build tools have been a key part in making testing available in a consistent manner everywhere.

NOTE You probably already know about how to unit test in Java with JUnit or another library. We will discuss other forms of testing in chapter 14.

Although compiling to class files is the start of a Java program's existence, generally, it isn't the end of the line. Fortunately, build tools also provide support for packaging your class files into a JAR or other format for easier distribution.

.1.2 Managing dependencies

In the early days of Java, if you wanted to use a library, you had to find its JAR somewhere, download the file, and put it into the classpath for your application. This caused several problems—in particular, the lack of a central, authoritative source for all libraries meant that a treasure hunt was sometimes necessary to find the JARs for less-common dependencies.

That obviously wasn't ideal, and so Maven (among other projects) gave the Java ecosystem repositories where tools could find and install dependencies for us. Maven Central remains to this day one of the most commonly used registries for Java dependencies on

the internet. Others also exist—public registries such as those hosted by Google or those shared on GitHub, and private installations via products such as Artifactory.

Downloading all that code can be time-consuming, too, so build tools have standardized on a few ways of reducing the pain by sharing artifacts between projects. With a local repository to cache, if a second project needs the same library, you don't need to download it again, as shown in figure 11.1. This approach also saves disk space, of course, but the single source of artifacts is the real win here.

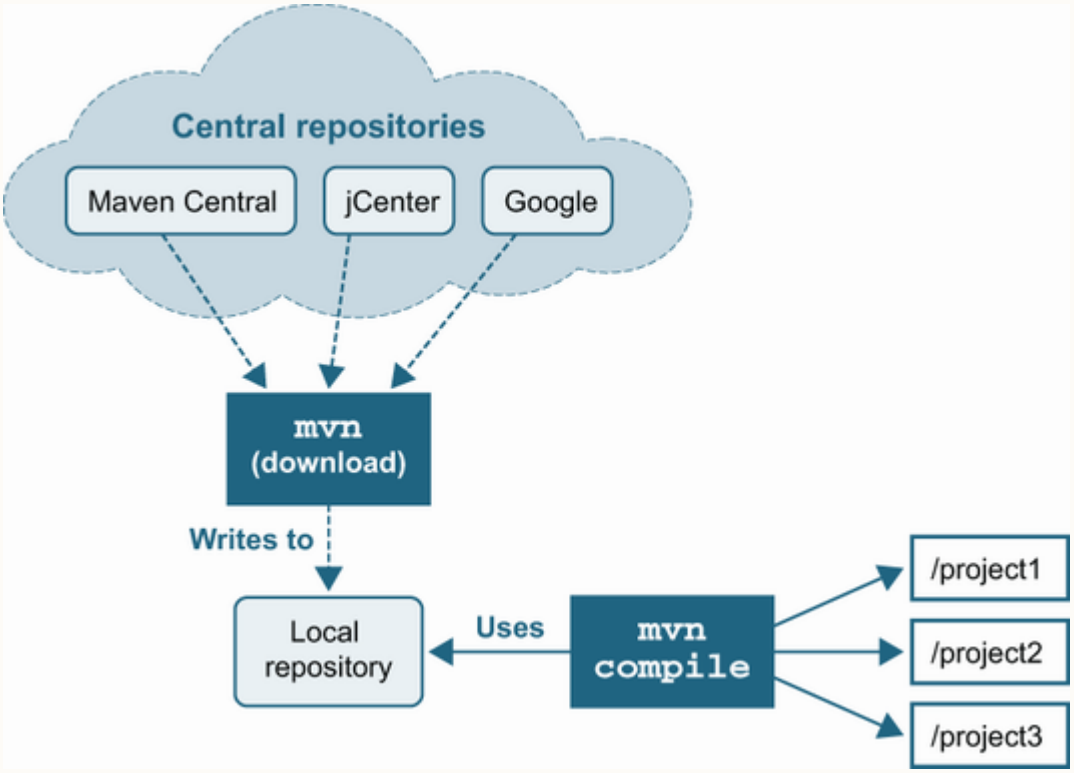


Figure 11.1 Maven's local repository helping not only to find dependencies online but to manage them efficiently locally

NOTE You might be wondering where modules fit in this dependency landscape. Modularized libraries are shipped as JAR files with the addition of the `module-info.class` file, as we saw in chapter 2. A modularized JAR can be downloaded from the standard repositories. The real differences come into play when you start compiling and running with modules, not in the packaging and distribution.

More than just providing a central place to find and download dependencies, though, registries opened the door for better management of *transitive dependencies*. In Java, we commonly see this situation when a library that our project uses itself depends on *another* library. We actually already met transitive dependency of modules in chapter 2, but the problem existed long before Java modules. In fact, before modules, the problem was significantly worse.

Recall that JAR files are just a zipped file—they don't have any metadata that describes the dependencies of the JAR. This means that the dependencies of a JAR are just the union of all of the dependencies of all the classes in the JAR.

To make matters worse, the class file format does not describe which version of a class is needed to satisfy the dependency—all we have is a symbolic descriptor of the class or method name that the class requires to link (as we saw in chapter 4). This implies the following two things:

1. An external source of dependency information is required.
2. As projects get larger, the transitive dependency graph will get increasingly complex.

With the explosion of open source libraries and frameworks to support developers, the typical tree of transitive dependencies in a real project has only gotten larger and larger.

One potential bit of good news is that the situation for the JVM ecosystem is somewhat better than it is for, say, JavaScript. JavaScript lacks a rich, central runtime library that is guaranteed to be always present, so a lot of basic capabilities have to be managed as external dependencies. This introduces problems such as multiple incompatible libraries that each provide a version of a common feature and a fragile ecosystem where mistakes and hostile attacks can have a disproportionate impact on the commons (e.g., the "left-pad" incident from 2016 [see **http://mng.bz/5Q64**]).

Java, on the other hand, has a runtime library (the JRE) that contains a lot of commonly needed classes, and this is available in every Java environment. However, a real production application will require capabilities beyond those in the JRE and will almost always have too many layers of dependencies to comfortably manage manually. The only solution is to automate.

A conflict emerges

This automation is a boon for developers building on the rich ecosystem of open source code available, but upgrading dependencies often reveals problems as well. For instance, figure 11.2 shows a dependency tree that might set us up for trouble.

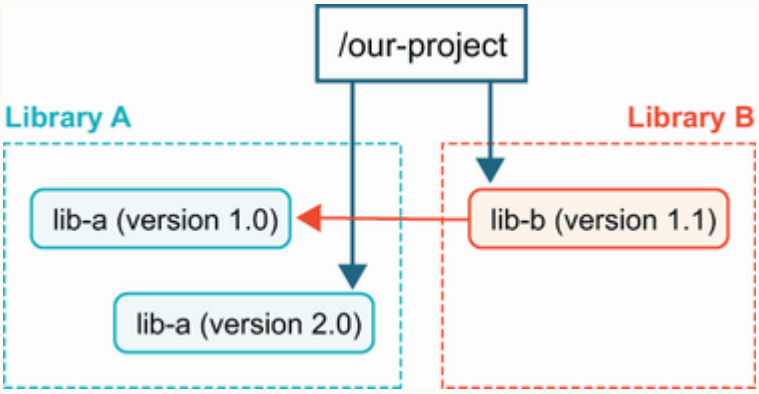


Figure 11.2 Conflicting transitive dependencies

We've asked explicitly for version 2.0 of `lib-a`, but our dependency `lib-b` has asked for the older version 1.0. This is known as a *dependency conflict*, and depending on how it is resolved, it can cause a variety of other problems.

What types of breakage can result from mismatched library versions? This depends on the nature of the changes between the versions. Changes fall into a few categories, shown here:

1. Stable APIs where only the behavior changes between versions
2. Added APIs where new classes or methods appear between versions
3. Changed APIs where method signatures or interfaces extended changes between versions
4. Removed APIs where classes or methods are removed between versions

In the case of a) or b), you may not even notice which version of the dependency your build tool has chosen. The most common case of c) is a change to the signature of a method between library versions. In our previous example, if `lib-a` 2.0 altered the signature of a method that `lib-b` relied upon, when `lib-b` tried to call that method, it would receive a `NoSuchMethodError` exception.

Removed methods in case d) would result in the same sorts of `NoSuchMethodError`. This includes "renaming" a method, which at the bytecode level isn't any different from removing a method and adding a new one that just happens to have the same implementation.

Classes are also prone to d) on deletion or renaming and will cause a `NoClassDefFoundError`. It's also possible that removal of interfaces from a class could land you with an ugly `ClassCastException`.

This list of issues with conflicting transitive dependencies is by no means exhaustive. It all boils down to *what* actually changes between two versions of the same package.

In fact, communicating about the nature of changes between versions is a common problem across languages. One of the most broadly adopted approaches to handling the problem is *semantic versioning* (see **https://semver.org/**). Semantic versioning gives us a vocabulary for stating the requirements of our transitive dependencies, which in turn allows the machines to help us sort them out.

When using semantic versioning, keep in mind the following:

- *MAJOR* version increments (1.x -> 2.x) on breaking changes to your API, like cases c) and d) above.
- *MINOR* version increments (1.1 -> 1.2) on backward-compatible additions like case b).
- *PATCH* increments on bug fixes (1.1.0 -> 1.1.1).

Though not foolproof, it at least gives an expectation as to what level of changes come with a version update and is broadly used in open source.

Having gotten a taste of why dependency management isn't easy, rest assured that both Maven and Gradle provide tooling to help. Later in the chapter, we'll look in detail at what each tool provides to unravel problems when you hit dependency conflicts.

.1.3 Ensuring consistency between developers

As projects grow in volume of code and developers involved, they often get more complex and harder to work with. Your build tooling can lessen this pain, though. Built-in features like ensuring everyone is compiling and running the same tests are a start. But we should consider many additions beyond the basics as well.

Tests are good, but how certain are you that *all* your code is tested? Code coverage tools are key for detecting what code is hit by your tests and what isn't. Although arguments swirl on the internet about the right target for code coverage, the line-level output coverage tools provide can save you from missing a test for that one extra special conditional.

Java as a language also lends itself well to a variety of static analysis tools. From detecting common patterns (i.e., overriding `equals` without overriding `hashCode`) to sniffing out unused variables, static analysis lets a computer validate aspects of the code that are legal but will bite you in production.

Beyond the realms of correctness, though, are style and formatting tools. Ever fought with someone about where the curly braces should go in a statement? How to indent your code? Agreeing once to a set of rules, even if they aren't all perfectly to your taste,

lets you focus forever after in the project on the actual work instead of nitpicking details about how the code looks.

Last and certainly not least, your build tool is a pivotal central point for providing custom functionality. Are there special setup or operational commands folks need to run periodically for your project? Validations your project should run after a build but before you deploy? All of these are excellent to consider wiring into the build tooling so they're available to everyone working with the code. Both Maven and Gradle provide many ways to extend them for your own logic and needs.

Hopefully you're now convinced that build tools aren't just something to set up once on a project but are worth the investment in understanding. Let's start by taking a look at one of the most common: Maven.

11.2 Maven

Early in Java history, the Ant framework was the default build tool. With tasks described in XML, it allowed a more Java-centric way to script builds than tools like Make. But Ant lacked structure around how to configure your build—-what the steps were, how they related, how dependencies were managed. Maven addressed many of these gaps with its concept of a standardized *build lifecycle* and a consistent approach to handling dependencies.

.2.1 The build lifecycle

Maven is an opinionated tool. One of the biggest areas where these opinions show is in its build lifecycles. Rather than users defining their own tasks and determining their order, Maven has a *default lifecycle* encompassing the usual steps, known as *phases*, that you'd expect in a build. Though not comprehensive, the following phases capture the high points in the default lifecycle:

- *Validate*—Check project configuration is correct and can build
- *Compile*—Compile the source code
- *Test*—Run unit tests
- *Package*—Generate artifacts such as JAR files
- *Verify*—Run integration tests
- *Install*—Install package to the local repository
- *Deploy*—Make package result available to others, typically run from a CI environment

Chances are that these map to most of the steps you'll take from source code to a deployed application or library. This is a major bonus to Maven's opinionated approach—any Maven project will share this same lifecycle. Your knowledge of how to run builds is more transportable than it used to be.

The phases are well-defined in Maven, but every project needs something special in the details. In Maven's model, various *plugins* attach *goals* to these phases. A goal is a concrete task, with the implementation of how to execute it.

Beyond the default lifecycle, Maven also includes the *clean* and *site* lifecycles. The *clean* lifecycle is intended for cleanup (e.g., removing intermediate build results), whereas the *site* lifecycle is meant for documentation generation.

We'll look closer at hooking into a lifecycle later when we discuss extending Maven, but if you truly need to redefine the universe, Maven does support authoring fully custom lifecycles. This is a very advanced topic, however, and beyond the scope of this book.

.2.2 Commands/POM intro

Maven is a project of the Apache Software Foundation and is open source. Installation instructions can be found on the project website at **https://maven.apache.org/install.html**.

Typically, Maven is installed globally on a developer's workstation, and it works on any not-ancient JVM (JDK 7 or higher). Once installed, invoking it gets us this output:

```
~: mvn

  [INFO] Scanning for projects...
  [INFO] ------------------------------------------------------------------
  [INFO] BUILD FAILURE
  [INFO] ------------------------------------------------------------------
  [INFO] Total time:  0.066 s
  [INFO] Finished at: 2020-07-05T21:28:22+02:00
  [INFO] ------------------------------------------------------------------
  [ERROR] No goals have been specified for this build. You must specify a
  valid lifecycle phase or a goal in the format <plugin-prefix>:<goal> or
  <plugin-group-id>:<plugin-artifact-id>[:<plugin-version>]:<goal>.
  Available lifecycle phases are: validate, initialize, ....
```

Of particular interest is the message that `No goals have been specified for this build.` This indicates that Maven doesn't know anything about our project. We provide that information in the `pom.xml` file, which is the center of the universe for a Maven project.

NOTE POM stands for Project Object Model.

Although a full-blown `pom.xml` file can be intimidatingly long and complex, you can get started with much less. For example, a more-or-less minimal `pom.xml` file looks like this:

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.wellgrounded</groupId>              (1)
  <artifactId>example</artifactId>                 (1)
  <version>1.0-SNAPSHOT</version>
  <name>example</name>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>   (2)
    <maven.compiler.target>11</maven.compiler.target>
  </properties>
</project>
```

(1) Identifying our project

(2) The Maven plugins default to Java 1.6. We obviously want a newer version.

Our `pom.xml` file declares two particularly important fields: the `groupId` and the `artifactId`. These fields combine with a version to form the *GAV coordinates* (group, artifact, version), which uniquely, globally identifies a specific release of your package. `groupId` typically specifies the company, organization, or open source project responsible for the library, whereas `artifactId` is the name for the specific library. GAV coordinates are often expressed with each part separated by a colon (`:`), such as `org.apache.commons:collections4:4.4` or `com.google.guava:guava:30.1-jre`.

These coordinates are important not just for configuring your project locally. Coordinates act as the address for dependencies, so our build tooling can find them. The following sections will dig into the mechanics of how we express those dependencies in more detail.

Much like Maven standardized the build lifecycle, it also popularized the standard layout we saw earlier in section 11.1.1 and shown next. If you follow these conventions, you don't have to tell Maven anything about your project for it to be able to compile:

```
.
├── pom.xml
└── src
    ├── main
    │   └── java
    │       └── com
    │           └── wellgrounded
    │               └── Main.java
    └── test
        └── java
            └── com
                └── wellgrounded
                    └── MainTest.java
```

Notice the parallel structures— `src/main/java` and `src/test/java` — with the same directories mapping to our package hierarchy. This convention keeps the test code separate from the main app code, which simplifies the process of packaging our main code for deployment, excluding the test code, which users of a package won't typically want or use.

Other standard directories exist beyond these two. For instance, `src/main/ resources` is the typical location for additional non-code files to include in a JAR. See the documentation at **http://mng.bz/6XoG** for a full listing of the Maven standard layout.

While you're getting used to Maven, it's a good idea to stick to the conventions, standard layouts, and other defaults that Maven provides. As we mentioned, it's an opinionated tool, so it's better to stay within the guardrails it provides while you're learning. Experienced Maven developers can (and do) stray outside the conventions and break the rules, but let's not try to run before we walk.

.2.3 Building

We saw previously that just running `mvn` on the command line warns us that we need to choose a lifecycle phase or goal to actually take action. Most often we'll want to run a phase, which may include many goals.

The simplest place to get started is compiling our code by requesting the `compile` phase like this:

```
~: mvn compile

  [INFO] Scanning for projects...
  [INFO]
  [INFO] ------------------< com.wellgrounded:example >---------------
  [INFO] Building example 1.0-SNAPSHOT
  [INFO] --------------------------[ jar ]-------------------------
  [INFO]
  [INFO] -- maven-resources-plugin:2.6:resources (default-resources) --
  [INFO] Using 'UTF-8' to copy filtered resources.                  (1)
  [INFO] Copying 0 resource
  [INFO]
  [INFO] ----- maven-compiler-plugin:3.1:compile (default-compile) ----
  [INFO] Changes detected - recompiling the module!                 (2)
  [INFO] Compiling 1 source file to ./maven-example/target/classes
  [INFO] ------------------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------------------
  [INFO] Total time:  0.940 s
  [INFO] Finished at: 2020-07-05T21:46:25+02:00
  [INFO] ------------------------------------------------------------
```

(1) Although we don't have resources in our project, the maven-resources-plugin from the default lifecycle checks for us.

(2) Our actual compilation is provided by maven-compiler-plugin.

Maven defaults our output to the `target` directory. After our `mvn compile`, we can find the class files under `target/classes`. Close inspection will reveal we only built the code under our `main` directory. If we want to compile our tests, you can use the `test-compile` phase.

The default lifecycle includes more than just compilation. For instance, `mvn package` for the previous project will result in a JAR file at `target/example-1.0-SNAPSHOT.jar`.

Although we can use this JAR as a library, if we try to run it via `java -jar target/ example-1.0-SNAPSHOT.jar`, we'll find that Java complains it can't find a main class. To see how we start growing our Maven build, let's change it so the produced JAR is a runnable application.

.2.4 Controlling the manifest

The JAR Maven produced from `mvn package` was missing a *manifest* to tell the JVM where to look for a `main` method on startup. Fortunately, Maven ships with a plugin for constructing JARs that knows how to write the manifest. The plugin exposes configuration via our `pom.xml` after the `properties` element and still inside the `project` element as follows:

```
<build>
 <plugins>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-jar-plugin</artifactId>          ❶
   <version>2.4</version>
   <configuration>                                    ❷
    <archive>
     <manifest>                                       ❸
      <addClasspath>true</addClasspath>
      <mainClass>com.wellgrounded.Main</mainClass>
      <Automatic-Module-Name>
       com.wellgrounded
      </Automatic-Module-Name>                        ❹
     </manifest>
    </archive>
   </configuration>
  </plugin>
 </plugins>
</build>
```

❶ maven-jar-plugin is the plugin name. You can spot this easily in the output when running the mvn package command.

❷ Each plugin has its own specialized configuration element with different child elements and attributes supported.

❸ <manifest> configures the resulting JAR's manifest contents.

❹ Configures our automatic module name

Adding this section sets up the main class so the `java` launcher knows how to directly execute the JAR. We have also added an automatic module name—this is to be good citizens in the modular world. As we discussed back in chapter 2, even if the code we're

writing is not modular (as in this case), it still makes sense to provide an explicit automatic module name so modular applications can more easily use our code.

This pattern of setting configuration under a `plugin` element is very standard in Maven. To simplify things, most default plugins will kindly warn if you use an unsupported or unexpected configuration property, although the details may vary by plugin.

.2.5 Adding another language

As we discussed in chapter 8, an advantage of the JVM as a platform is the ability to use multiple languages within the same project. This may be useful when a specific language has better facilities for a given part of your application, or even to allow gradual conversion of an application from one language to another.

Let's take a look at how we would configure our simple Maven project to build some classes from Kotlin instead of Java. Our standard layout is fortunately already set to allow for easy adding languages, as shown next:

```
.
├── pom.xml
└── src
    ├── main
    │   ├── java
    │   │   └── com
    │   │       └── wellgrounded
    │   │           └── Main.java
    │   └── kotlin                          (1)
    │       └── com
    │           └── wellgrounded            (2)
    │               └── MessageFromKotlin.kt
    └── test
        └── java
            └── com
                └── wellgrounded
                    └── MainTest.java
```

(1) We keep our Kotlin code in its own subdirectory so it's easy to tell what paths use which compiler to produce class files.

(2) Packages can mix between the languages, because the resulting class files don't have direct knowledge of what language they were generated from.

Unlike Java, Maven by default doesn't know how to compile Kotlin so we need to add `kotlin-maven-plugin` in our `pom.xml`. We recommend consulting the Kotlin documentation at **https://kotlinlang.org/docs/maven.html** for the most up-to-date usage, but we'll demonstrate here so you can know what to expect.

If a project is fully written in Kotlin, compilation only needs the plugin added and attached to the `compile` goal as follows:

```
<build>
 <plugins>
  <plugin>
   <groupId>org.jetbrains.kotlin</groupId>
   <artifactId>kotlin-maven-plugin</artifactId>
   <version>1.6.10</version>                    (1)
   <executions>
    <execution>
     <id>compile</id>
     <goals>                                    (2)
      <goal>compile</goal>
     </goals>
    </execution>
    <execution>
     <id>test-compile</id>
     <goals>                                    (2)
      <goal>test-compile</goal>
     </goals>
    </execution>
   </executions>
  </plugin>
 </plugins>
</build>
```

(1) Current version of Kotlin, as of when this chapter was written.

(2) Adds this plugin to the goals for compiling main and test code.

The situation gets more complex when mixing Kotlin and Java. Maven's default `maven-compiler-plugin`, which compiles Java for us, needs to be overridden to let Kotlin compile first, as shown next, or our Java code will be unable to use the Kotlin classes:

```
<build>
 <plugins>
  <plugin>
```

```
 <groupId>org.jetbrains.kotlin</groupId>
 <artifactId>kotlin-maven-plugin</artifactId>                    (1)
 <version>1.6.10</version>
 <executions>
  <execution>
   <id>compile</id>
   <goals>
    <goal>compile</goal>
   </goals>
   <configuration>
    <sourceDirs>                                                  (2)
     <sourceDir>${project.basedir}/src/main/kotlin</sourceDir>
     <sourceDir>${project.basedir}/src/main/java</sourceDir>
    </sourceDirs>
   </configuration>
  </execution>
  <execution>
   <id>test-compile</id>
   <goals>
    <goal>test-compile</goal>
   </goals>
   <configuration>
    <sourceDirs>                                                  (2)
     <sourceDir>${project.basedir}/src/test/kotlin</sourceDir>
     <sourceDir>${project.basedir}/src/test/java</sourceDir>
    </sourceDirs>
   </configuration>
  </execution>
 </executions>
</plugin>
<plugin>
 <groupId>org.apache.maven.plugins</groupId>
 <artifactId>maven-compiler-plugin</artifactId>
 <version>3.8.1</version>
 <executions>
  <execution>
   <id>default-compile</id>                                       (3)
   <phase>none</phase>
  </execution>
  <execution>
   <id>default-testCompile</id>                                   (3)
   <phase>none</phase>
  </execution>
  <execution>
   <id>java-compile</id>                                          (4)
   <phase>compile</phase>
```

```
        <goals>
          <goal>compile</goal>
        </goals>
      </execution>
      <execution>
        <id>java-test-compile</id>                    ❹
        <phase>test-compile</phase>
        <goals>
          <goal>testCompile</goal>
        </goals>
      </execution>
    </executions>
  </plugin>
 </plugins>
</build>
```

❶ Adds the kotlin-maven-plugin mostly as before, making sure now it's aware of both Java and Kotlin paths

❷ The Kotlin compiler needs to know about both our Kotlin and Java code locations.

❸ Disables the maven-compiler-plugin defaults for building Java because these force it to run first

❹ Reapplies the maven-compiler-plugin to the compile and test-compile phases. These will now be added after the kotlin-maven-plugin.

NOTE The above overrides may get complicated when using Maven features like parent projects, where additional POM definitions might come into conflict. We'll see some tactics soon for debugging when these issues arise.

Your project will need a dependency on at least the Kotlin standard library, so we add that explicitly like so:

```
<dependencies>
  <dependency>
    <groupId>org.jetbrains.kotlin</groupId>
    <artifactId>kotlin-stdlib</artifactId>
    <version>1.6.10</version>
  </dependency>
</dependencies>
```

With this in place, our multilingual project builds and runs as before.

.2.6 Testing

Once your code builds, a smart next step is to test it. Maven integrates testing deeply into its lifecycle. In fact, where compilation of your main code is only a single phase, Maven supports two separate phases of testing out of the box: `test` and `integration-test`. `test` is used for typical unit testing, whereas the `integration-test` phase runs after construction of artifacts such as JARs, with the intent of performing end-to-end validation on your final outputs.

NOTE Integration tests may also be run with JUnit because, despite the name, JUnit is a very capable test runner for more than just unit testing. Do not fall into the trap of thinking that any test executed by JUnit is automatically a unit test! We'll examine the different types of tests in detail in chapter 13.

Almost any project will benefit from some testing. As you might expect from Maven's opinionated stance, testing happens (by default) using the near ubiquitous framework JUnit. Other frameworks are just a plugin away.

Although the standard plugins know about running JUnit, we still must declare the library as a dependency so Maven knows how to compile our tests. You can add a library with a snippet like the following under the `<project>` element:

```
<dependencies>
 <dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.8.1</version>
  <scope>test</scope>                    (1)
 </dependency>
 <dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <version>5.8.1</version>
  <scope>test</scope>                    (1)
 </dependency>
</dependencies>
```

(1) <scope> indicates this library is needed only for the test-compile phase.

With that in place, we can try to run our unit tests. Depending on your version of Maven, even the most recent versions may give us this odd result:

```
~:mvn test

 [INFO] Scanning for projects...
 [INFO]
 [INFO] -------------------< com.wellgrounded:example >----------------
 [INFO] Building example 1.0-SNAPSHOT
 [INFO] --------------------------------[ jar ]---------------------
 [INFO]
 [INFO] .....
 [INFO]
 [INFO] -- maven-surefire-plugin:2.12.4:test (default-test) @ example -
 [INFO] Surefire report dir: ./target/surefire-reports                  (1)

 -------------------------------------------------------
  T E S T S
 -------------------------------------------------------
 Running com.wellgrounded.MainTest
 Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 sec

 Results :

 Tests run: 0, Failures: 0, Errors: 0, Skipped: 0                       (2)

 [INFO] ------------------------------------------------
 [INFO] BUILD SUCCESS
 [INFO] ------------------------------------------------
 [INFO] Total time:  5.605 s
 [INFO] Finished at: 2021-11-29T09:41:06+01:00
 [INFO] ------------------------------------------------
```

(1) Maven's default for running JUnit tests is the maven-surefire-plugin.

(2) No tests were run? That's not right!

For compatibility reasons, the plugin `maven-surefire-plugin` that is installed by default, even as late as Maven 3.8.4, isn't aware of JUnit 5. We'll dig more into these conversion issues in chapter 13, but in the meantime, let's just bump our version of the plugin to something more recent, as shown here:

```
<plugin>
 <groupId>org.apache.maven.plugins</groupId>
 <artifactId>maven-surefire-plugin</artifactId>
 <version>3.0.0-M5</version>                    (1)
</plugin>
```

(1) Moving later than 2.12, the plugins understand JUnit 5 directly.

With that in place we see the following more reassuring outcome:

```
~:mvn test

 [INFO] .....

 -------------------------------------------------------
  T E S T S
 -------------------------------------------------------
 Running com.wellgrounded.MainTest
 Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.04 sec

 Results :

 Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

 [INFO] ------------------------------------------------
 [INFO] BUILD SUCCESS
 [INFO] ------------------------------------------------
 [INFO] Total time:  1.010 s
 [INFO] Finished at: 2020-07-06T15:45:22+02:00
 [INFO] ------------------------------------------------------------
```

By default, the Surefire plugin runs all unit tests in the standard location, `src/test/*`, during the `test` phase. If we want to take advantage of the `integration-test` phase, it's recommended to use a separate plugin, such as `maven-failsafe-plugin`. Failsafe is maintained by the same folks who make `maven-surefire-plugin` and specifically targets the integration testing case. We add the plugin in our `<build><plugins>` section we previously used for configuring our manifest as follows:

```
<plugin>
 <groupId>org.apache.maven.plugins</groupId>
 <artifactId>maven-failsafe-plugin</artifactId>
 <version>3.0.0-M5</version>
```

```
    <executions>
     <execution>
      <goals>
       <goal>integration-test</goal>
       <goal>verify</goal>
      </goals>
     </execution>
    </executions>
   </plugin>
```

Failsafe treats the following filename patterns as integration tests, although it can be reconfigured:

- `**/IT*.java`
- `**/*IT.java`
- `**/*ITCase.java`

Because it's part of the same suite of plugins, Surefire is also aware of this convention and excludes these tests from the `test` phase.

It's recommended to run integration tests via `mvn verify`, as shown next, rather than `mvn integration-test`. `verify` includes `post-integration-test`, which is the typical location for plugins to attach any post-test cleanup work if any is needed:

```
~: mvn verify

 [INFO] ... compilation output omitted for length ...

 [INFO] --- maven-failsafe-plugin:3.0.0-M5:integration-test @ example ---
 [INFO]
 [INFO] -------------------------------------------------------
 [INFO]  T E S T S
 [INFO] -------------------------------------------------------
 [INFO] Running com.wellgrounded.LongRunningIT
 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0,
 [INFO] Time elapsed: 0.032 s - in com.wellgrounded.LongRunningIT
 [INFO]
 [INFO] Results:
 [INFO]
 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
 [INFO]
 [INFO]
 [INFO] --- maven-failsafe-plugin:3.0.0-M5:verify (default) @ example ---
```

```
[INFO] ------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------
```

.2.7 Dependency management

A key feature Maven brought to the ecosystem was a standard format for expressing dependency management information via the `pom.xml` file. Maven also established a central repository for libraries. Maven can walk your `pom.xml` and the `pom.xml` files from your dependencies to determine the entire set of *transitive dependencies* your application requires.

The process of walking the tree and finding all the necessary libraries is called *dependency resolution*. Though critical for managing modern applications, the process does have its sharp edges.

To see where the problems arise, let's revisit the project setup we saw earlier in section 11.1.2. Recall that the project's dependencies have resulted in a tree that looks like that shown in figure 11.3.

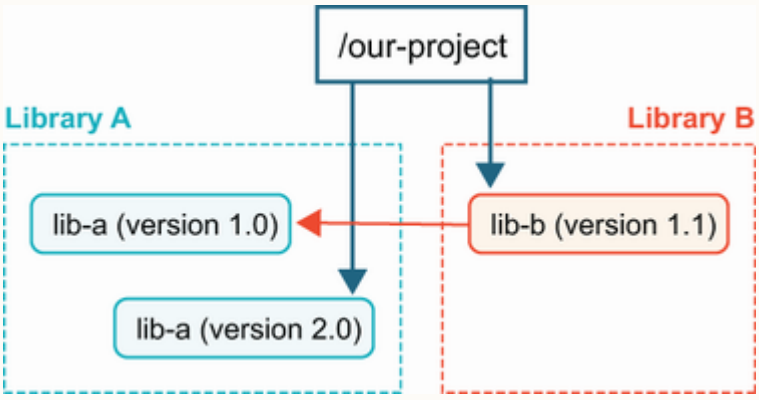


Figure 11.3 Conflicting transitive dependencies where a dependency requests an older version

Here we've asked explicitly for version 2.0 of `lib-a`, but our dependency `lib-b` has asked for the older version 1.0. Maven's dependency resolution algorithm favors the version of a library closest to the root. The end result of the configuration shown in figure 11.3 is that we will use `lib-a` 2.0 in our application. As we outlined in section 11.1.2, this may work fine or be disastrously broken.

Another common scenario that can also cause problems is when the reverse occurs and the dependency that is closest to the root is *older* than the one expected as a transitive dependency, as depicted in figure 11.4.

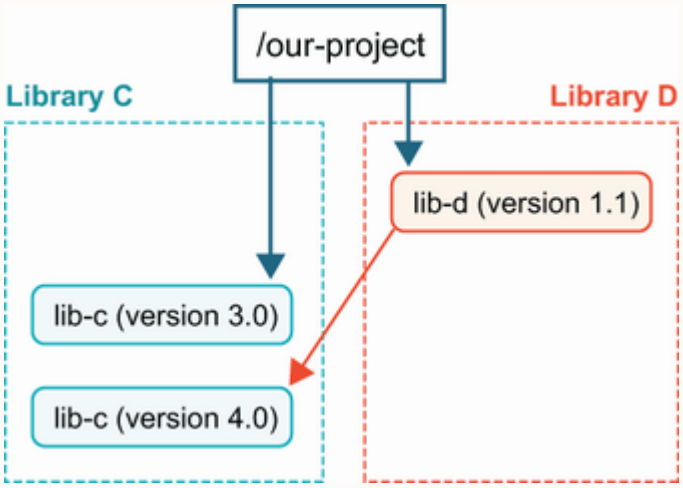


Figure 11.4 Conflicting transitive dependencies where a dependency asks for a newer version

In this case, it's entirely possible that `lib-d` is relying on an API in `lib-c` that didn't exist in version 3.0, so adding a dependency on `lib-d` to a project that is already using `lib-c` will result in runtime exceptions.

NOTE Given those possibilities, we recommended any package your code directly interacts with should be declared explicitly in your `pom.xml`. If you don't, and instead rely upon transitive dependency, updating your direct dependency could result in unexpected build breakage.

Before we can solve our dependency problems, it's important to know what our dependencies are. Maven has us covered with the `mvn dependency:tree` command, shown here:

```
~:mvn dependency:tree
  [INFO] Scanning for projects...
  [INFO]
  [INFO] -------------------< com.wellgrounded:example >---------------
  [INFO] Building example 1.0-SNAPSHOT
  [INFO] --------------------------------[ jar ]---------------------------
  [INFO]
  [INFO] -- maven-dependency-plugin:2.8:tree (default-cli) @ example --
  [INFO] com.wellgrounded:example:jar:1.0-SNAPSHOT
  [INFO] +- org.junit.jupiter:junit-jupiter-api:jar:5.8.1:test
  [INFO] |  +- org.opentest4j:opentest4j:jar:1.2.0:test
  [INFO] |  +- org.junit.platform:junit-platform-commons:jar:1.8.1:test
  [INFO] |  \- org.apiguardian:apiguardian-api:jar:1.1.2:test
  [INFO] \- org.junit.jupiter:junit-jupiter-engine:jar:5.8.1:test
  [INFO]    \- org.junit.platform:junit-platform-engine:jar:1.8.1:test
  [INFO] ------------------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------------------
```

```
[INFO] Total time:  0.790 s
[INFO] Finished at: 2020-08-13T23:02:10+02:00
[INFO] ------------------------------------------------------------
```

The tree from this command shows us our direct dependencies on JUnit from the `pom.xml` file at the first level of nesting, followed by JUnit's own transitive dependencies.

JUnit comes with a slim set of dependencies, so to explore transitive dependency issues further, let's imagine that our team wants to use two internal libraries at our company to get support for doing custom assertions. These are both built using the `assertj` library, but unfortunately different versions, as shown next:

```
[INFO] com.wellgrounded:example:jar:1.0-SNAPSHOT
[INFO] +- org.junit.jupiter:junit-jupiter-api:jar:5.8.1:test
[INFO] |  +- org.opentest4j:opentest4j:jar:1.2.0:test
[INFO] |  +- org.junit.platform:junit-platform-commons:jar:1.8.1:test
[INFO] |  \- org.apiguardian:apiguardian-api:jar:1.1.2:test
[INFO] +- org.junit.jupiter:junit-jupiter-engine:jar:5.8.1:test
[INFO] |  \- org.junit.platform:junit-platform-engine:jar:1.8.1:test
[INFO] +- com.wellgrounded:first-test-helper:1.0.0:test
[INFO] |  \- org.assertj:assertj-core:3.21.0:test          (1)
[INFO] \- com.wellgrounded:second-test-helper:2.0.0:test
[INFO]    \- org.assertj:assertj-core:2.9.1:test          (2)
```

(1) Our first helper library brings assertj-core with version 3.21.0.

(2) Our second helper library wants assertj-core with version 2.9.1.

The best possible approach is finding newer versions of our dependencies that can all agree on their dependencies. As internal libraries, this is obviously a possibility. Even in the broader world of open source, it's often possible. Having said that, sometimes libraries lose their maintainers and fall out of date, so it is entirely possible to get stuck in a situation where it's difficult to get the update we desire.

This leaves us looking for other ways to deal with the conflict. Two main approaches come into play if we can't find a natural resolution. Be aware that both of these solutions require finding *some* compatible version that will satisfy your dependencies.

If one of your dependencies specifies a version that everyone could agree on, but it isn't being chosen by Maven's resolution algorithm, you can tell Maven to exclude parts of the tree when resolving. If both of our helper libraries can work fine with the newer `as-`

sertj-core , we can just ignore the older one brought by the second library, as shown here:

```
<dependencies>
  <dependency>
   <groupId>com.wellgrounded</groupId>
   <artifactId>second-test-helper</artifactId>
   <version>2.0.0</version>
   <scope>test</scope>
   <exclusions>                    (1)
    <exclusion>
     <groupId>org.assertj</groupId>
     <artifactId>assertj-core</artifactId>
    </exclusion>
   </exclusions>
  </dependency>
  <dependency>
   <groupId>com.wellgrounded</groupId>      (2)
   <artifactId>first-test-helper</artifactId>
   <version>1.0.0</version>
   <scope>test</scope>
  </dependency>
 </dependencies>
```

(1) Excludes the second-test-helper obsolete version of assertj-core

(2) Lets the transitive dependency from first-test-helper proceed normally

In the worst case, perhaps neither library expresses the compatible version. To handle this, we specify the precise version as a direct dependency in our project, as shown in the following code sample. By its resolution rules, Maven will choose that version because it is closer to the project root. Although this convinces the tool to do what we want, we are taking on the risk of runtime errors from mixing libraries versions, so it is important to test the interactions thoroughly:

```
<dependencies>
  <dependency>
   <groupId>com.wellgrounded</groupId>
   <artifactId>second-test-helper</artifactId>   (1)
   <version>2.0.0</version>
   <scope>test</scope>
  </dependency>
  <dependency>
```

```
    <groupId>com.wellgrounded</groupId>
    <artifactId>first-test-helper</artifactId>    ❷
    <version>1.0.0</version>
    <scope>test</scope>
   </dependency>
   <dependency>
    <groupId>org.assertj</groupId>
    <artifactId>org.assertj</artifactId>
    <version>3.1.0</version>                      ❸
    <scope>test</scope>
   </dependency>
  </dependencies>
```

❶ Our dependencies will ask for assertj-core at a different version.

❷ Our dependencies will ask for assertj-core at a different version.

❸ But we force resolution on assertj-core to the precise version we want.

Finally, it's worth noting that the `maven-enforcer-plugin` can be configured to fail the build if any mismatched dependencies are found so we can avoid relying on bad runtime behavior to surface problems. (See **http://mng.bz/o2WN**.) These build failures can then be addressed using the techniques we've discussed earlier.

.2.8 Reviewing

Our build process is an excellent spot to hook in additional tooling and checks. One key bit of information is code coverage, which informs us what parts of our code our tests execute.

A leading option for code coverage in the Java ecosystem is JaCoCo (**http://mng.bz/nNjv**). JaCoCo can be configured to enforce certain coverage levels during testing and will output reports that tell you what is and isn't covered.

Enabling JaCoCo requires only adding a plugin to the `<build><plugins>` section of your `pom.xml` file. It doesn't enable itself by default, so you have to tell it when it should execute. In this example we've bound it to the `test` phase like this:

```
  <build>
   <plugins>
    <plugin>
     <groupId>org.jacoco</groupId>
```

```
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.5</version>
      <executions>
       <execution>                    (1)
        <goals>
         <goal>prepare-agent</goal>
        </goals>
       </execution>
       <execution>                    (2)
        <id>report</id>
        <phase>test</phase>
        <goals>
         <goal>report</goal>
        </goals>
       </execution>
      </executions>
     </plugin>
    </plugins>
   </build>
```

(1) JaCoCo needs to start running early in the process. This adds it to the initialize phase.

(2) Tells JaCoCo to report during the test phase

This produces reports on all of your classes in `target/site/jacoco` by default, as shown in figure 11.5, with a full HTML version at `index.html` to be explored.

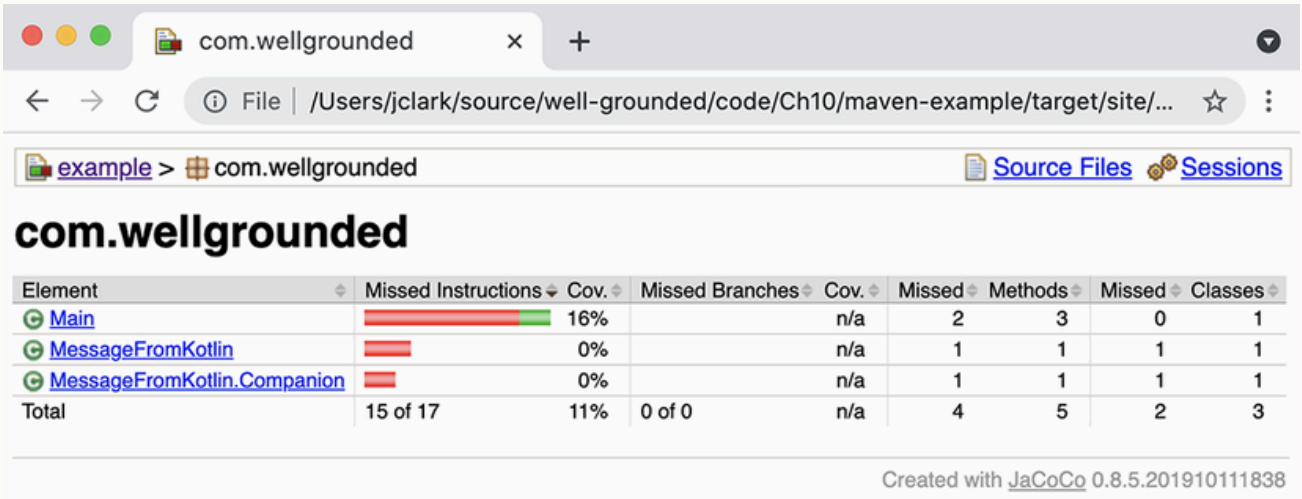


Figure 11.5 JaCoCo coverage report page

.2.9 Moving beyond Java 8

In chapter 1, we noted the following series of modules that belonged with Java Enterprise Edition but were present in the core JDK. These were deprecated with JDK 9 and removed with JDK 11 but remain available as external libraries:

- `java.activation` (JAF)
- `java.corba` (CORBA)
- `java.transaction` (JTA)
- `java.xml.bind` (JAXB)
- `java.xml.ws` (JAX-WS, plus some related technologies)
- `java.xml.ws.annotation` (Common Annotations)

If your project relies on any of these modules, your build might break when you move to a more recent JDK. Fortunately a few simple dependency additions in your `pom.xml`, shown here, address the issue:

```
<dependencies>
 <dependency>
  <groupId>com.sun.activation</groupId>          (1)
  <artifactId>jakarta.activation</artifactId>
  <version>1.2.2</version>
 </dependency>
 <dependency>
  <groupId>org.glassfish.corba</groupId>         (2)
  <artifactId>glassfish-corba-omgapi</artifactId>
  <version>4.2.1</version>
 </dependency>
 <dependency>
  <groupId>javax.transaction</groupId>           (3)
  <artifactId>javax.transaction-api</artifactId>
  <version>1.3</version>
 </dependency>
 <dependency>
  <groupId>jakarta.xml.bind</groupId>            (4)
  <artifactId>jakarta.xml.bind-api</artifactId>
  <version>2.3.3</version>
 </dependency>
 <dependency>
  <groupId>jakarta.xml.ws</groupId>              (5)
  <artifactId>jakarta.xml.ws-api</artifactId>
  <version>2.3.3</version>
 </dependency>
 <dependency>
```

```
    <groupId>jakarta.annotation</groupId>          ❻
    <artifactId>jakarta.annotation-api</artifactId>
    <version>1.3.5</version>
  </dependency>
</dependencies>
```

❶ java.activation (JAF)

❷ java.corba (CORBA)

❸ java.transaction (JTA)

❹ java.xml.bind (JAXB)

❺ java.xml.ws (JAX-WS, plus some related technologies)

❻ java.xml.ws.annotation (Common Annotations)

.2.10 Multirelease JARs in Maven

A feature that arrived in JDK 9 was the ability to package JARs that target different code for different JDKs. This allows us to take advantage of new features in the platform, while still supporting clients of our code on older versions.

In chapter 2, we examined the feature and hand-crafted the specific JAR format necessary to enable this capability. The layout places versioned directories under `META-INF/versions` within the JAR where the JVM from 9 onward will check for newer versions of a given class during loading, as shown next:

```
.
├── META-INF
│   ├── MANIFEST.MF
│   └── versions
│       └── 11
│           └── wgjd2ed
│               └── GetPID.class
└── wgjd2ed
    ├── GetPID.class
    └── Main.class
```

Within this structure, the classes in `wgjd2ed` will have a class file version representing the oldest JVM the JAR may be used with. (In our later example, this will be JDK 8.)

Classes under `META-INF/versions/11`, though, may be compiled with a newer JDK and have a newer class file version. Because older JDKs ignore the `META-INF/versions` directory (and those from 9 onward understand what versions they're allowed to use), we can mix newer code in a JAR while still having everything work on an older JVM. This is exactly the sort of tedious process that Maven was built to automate.

Although the output format in our JAR is all that really matters to enable the multirelease feature, we'll mimic the structure in our code layout for clarity. As shown here, the code in `src` is the baseline functionality that will be seen by any JDK by default. The code under `versions` optionally replaces specific classes with an alternate implementation:

```
.
├── pom.xml
├── src
│   └── main
│       └── java
│           └── wgjd2ed
│               └── GetPID.java
│                   └── Main.java
└── versions
    └── 11
        └── src
            └── wgjd2ed
                └── GetPID.java
```

Maven's defaults will find and compile our code in `src/main`, but we have two complications we need to sort out:

- Maven needs to *also* find our code in the `versions` directory.
- Further, Maven needs to compile that source targeted to a different JDK than the main project.

Both of these goals can be accomplished by configuring the `maven-compiler-plugin` that builds our Java class files. We introduce two separate `<execution>` steps in the next code snippet—one to compile the base code targeting JDK 8, and then a second pass to compile the versioned code targeting JDK 11.

NOTE We must compile using a JDK version at least as new as the latest version you're targeting. However, we'll explicitly instruct some build steps to target a lower version than the compiler is capable of.

```
<plugins>
 <plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.1</version>
  <executions>
   <execution>
    <id>compile-java-8</id>                    (1)
    <goals>
     <goal>compile</goal>
    </goals>
    <configuration>
     <source>1.8</source>                      (2)
     <target>1.8</target>
    </configuration>
   </execution>
   <execution>
    <id>compile-java-11</id>                   (3)
    <phase>compile</phase>
    <goals>
     <goal>compile</goal>
    </goals>
    <configuration>
     <compileSourceRoots>                      (4)
      <compileSourceRoot>
       ${project.basedir}/versions/11/src
      </compileSourceRoot>
     </compileSourceRoots>
     <release>11</release>                     (5)
     <multiReleaseOutput>                      (5)
      true
     </multiReleaseOutput>
    </configuration>
   </execution>
  </executions>
 </plugin>
</plugins>
```

(1) Execution step to compile for JDK 8

(2) We'll compile with JDK 11, so we target this step's output to JDK 8.

❸ Second execution step for targeting JDK 11

❹ Tells Maven about our alternate location for the version-specific code

❺ Setting release and multiReleaseOutput tells Maven which JDK this versioned code is intended for and asks it to put the classes at the correct multirelease location in output.

This gets our JAR built and packaged with the right layout. There's one more step, and that's marking the manifest as multirelease. This is configured in the `maven-jar-plugin`, as shown here, close to where we made our application JAR executable in section 11.2.4:

```
<plugin>
 <groupId>org.apache.maven.plugins</groupId>
 <artifactId>maven-jar-plugin</artifactId>
 <version>3.2.0</version>
 <configuration>
  <archive>
   <manifest>
    <addClasspath>true</addClasspath>
    <mainClass>wgjd2ed.Main</mainClass>
   </manifest>
   <manifestEntries>                    ❶
    <Multi-Release>true</Multi-Release>
   </manifestEntries>
  </archive>
 </configuration>
</plugin>
```

❶ Attribute to mark the JAR as multirelease

With that we can execute our code against different JDKs and see it behave as expected. In the case of our sample app, the base implementation for JDK 8 will output an additional version message, as illustrated here, so we can see it's working:

```
~:mvn clean compile package
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------< wgjd2ed:maven-multi-release >-------------------
[INFO] Building maven-multi-release 1.0-SNAPSHOT
[INFO] --------------------------[ jar ]---------------------------------
[INFO]
[INFO] .... Lots of additional steps
[INFO]
```

```
[INFO] - maven-jar-plugin:3.2.0:jar (default-jar) @ maven-multi-release -
[INFO] Building jar: ~/target/maven-multi-release-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------
[INFO] Total time:  1.813 s
[INFO] Finished at: 2021-03-05T09:39:16+01:00
[INFO] ------------------------------------------------------------------

~:java -version
openjdk version "11.0.6" 2020-01-14
OpenJDK Runtime Environment AdoptOpenJDK (build 11.0.6+10)
OpenJDK 64-Bit Server VM AdoptOpenJDK (build 11.0.6+10, mixed mode)

~:java -jar target/maven-multi-release-1.0-SNAPSHOT.jar
75891

# Change JDK versions by your favorite means....

~:java -version
openjdk version "1.8.0_265"
OpenJDK Runtime Environment (AdoptOpenJDK)(build 1.8.0_265-b01)
OpenJDK 64-Bit Server VM (AdoptOpenJDK)(build 25.265-b01, mixed mode)

~:java -jar target/maven-multi-release-1.0-SNAPSHOT.jar
Java 8 version...
76087
```

The path to using new features in the JDK without abandoning older clients is all set!

.2.11 Maven and modules

In chapter 2, we examined the JDK's new module system in detail. Let's look at how it influences our build scripting. We'll start looking at a simple library that exposes one of its packages publicly while hiding the other.

A MODULAR LIBRARY

Modular projects vary slightly in their code layout from the strict Maven standard. The `main` directory instead reflects the name of the module, as shown here:

```
.
├── pom.xml
└── src
```

```
└── com.wellgrounded.modlib                    ❶
    └── java
        └── com
            └── wellgrounded
                ├── hidden
                │   └── CantTouchThis.java     ❷
                └── visible
                    └── UseThis.java           ❸
```

❶ Our modular code directory

❷ A class we intend to keep private

❸ A class we intend to share publicly through our module

Having made that change, we have to inform Maven of this new location to look for source code to compile as follows:

```
<build>
  <sourceDirectory>src/com.wellgrounded.modlib/java</sourceDirectory>
</build>
```

The final piece to making our library modular is the addition of a `module-info.java` at the root of our code (alongside the `com` directory). This will name our module, and declare what we allow access to, as shown here:

```
module com.wellgrounded.modlib {
   exports com.wellgrounded.modlib.visible;
}
```

Everything else about this simple library remains the same, and if we `mvn package`, we'll get a JAR file in `target`. Before we proceed further, we can also put this library into the local Maven cache via `mvn install`.

NOTE The JDK's module system is about access control at build and runtime, *not* packaging. A modular library can be shared as a plain old JAR file, just with the additional `module-info.class` included to tell modular applications how to interact with it.

Now that we have a modular library, let's build a modular application to consume it.

Our modular application gets a similar layout to what we used for the library, as shown next:

```
.
├── pom.xml
└── src
    └── com.wellgrounded.modapp
        └── java
            ├── com
            │   └── wellgrounded
            │       └── Main.java
            └── module-info.java
```

Our `module-info.java` for the application declares our name, and states that we require the package exported by our library as follows:

```
module com.wellgrounded.modapp {
  requires com.wellgrounded.modlib;
}
```

This by itself doesn't tell Maven where to find our library JAR, though, so we include it as a normal `<dependency>` like this:

```
<dependencies>
  <dependency>
    <groupId>com.wellgrounded</groupId>  (1)
    <artifactId>modlib</artifactId>
    <version>2.0</version>
  </dependency>
 </dependencies>
```

(1) Our library from the prior section, installed into the local Maven repository

When we're compiling and subsequently running, it's important that this dependency be placed on the module path instead of the classpath. How does Maven accomplish this? Fortunately, recent versions of the `maven-compiler-plugin` are smart enough to notice that 1) our application has a `module-info.java`, so it's modular; and 2) the dependency includes `module-info.class`, so it, too, is a module. As long as you are on a recent ver-

sion of `maven-compiler-plugin` (3.8 worked great at the time of writing), Maven figures it out for you.

Our application code is perfectly normal Java, and we can use the modular library's functionality as intended, as follows:

```
package com.wellgrounded.modapp;

import com.wellgrounded.modlib.visible.UseThis;  (1)

public class Main {
  public static void main(String[] args) {
    System.out.println(UseThis.getMessage());  (2)
  }
}
```

(1) import from the module, just like any other package.

(2) Uses the class from our module to get a message

You may remember that we had another package in our library that we didn't provide access to. What happens if we modify our application to try and pull that in, like so:

```
package com.wellgrounded.modapp;

import com.wellgrounded.modlib.visible.UseThis;
import com.wellgrounded.modlib.hidden.CantTouchThis;  (1)

public class Main {
  public static void main(String[] args) {
    System.out.println(UseThis.getMessage());
  }
}
```

(1) com.wellgrounded.modlib.hidden was not listed in the library's exports.

Compiling this will give us the following error straight away:

```
[INFO] - maven-compiler-plugin:3.8.1:compile @ modapp ---
  [INFO] Changes detected - recompiling the module!
  [INFO] Compiling 2 source files to /mod-app/target/classes
```

```
[INFO] -------------------------------------------------------------
[ERROR] COMPILATION ERROR :
[INFO] -------------------------------------------------------------
[ERROR]                                                          (1)
  src/com.wellgrounded.modapp/java/com/wellgrounded/Main.java:[4,31]
    package com.wellgrounded.modlib.hidden is not visible (package
    com.wellgrounded.modlib.hidden is declared in module
    com.wellgrounded.modlib, which does not export it)

[INFO] 1 error
[INFO] -------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] -------------------------------------------------------------
```

(1) javac and the module system won't even let us try to touch things that aren't exported!

Maven's tooling has come a long way since the release of modules in JDK 9. All the standard scenarios are well covered with a minimum of additional configuration required.

Before we go, though, let's take one brief tangent. Throughout this section, `module-info.class` was frequently the signal to Maven that it should start applying modular rules. But modules are an *opt-in* feature in the JDK to preserve compatibility with the vast quantities of premodular code out there.

What happens if we build the same application using our modular library, but the application doesn't mark itself to use modules by including the `module-info.java` file? In that case, the library—although it is modular—will be included via the classpath. This places it in the unnamed module along with the application's own code, and all those access restrictions we defined in the library are effectively ignored. A sample application is included in the supplement alongside the modular one that uses our library by classpath so you can see more clearly how opting into or out of modules works.

With that, our tour of Maven's default features is done. But what do we do if we need to extend the system beyond the admittedly vast array of plugins we can find online?

.2.12 Authoring Maven plugins

Even the most basic defaults in Maven are supplied as plugins, and there's no reason you can't write one, too, when we need to do more. As we've seen, referencing a plugin is a lot like pulling in a dependent library. It isn't surprising, then, that we implement our Maven plugins as separate JAR files.

For our example, we start with a `pom.xml` file. Much of the boilerplate is similar to before with a couple small additions, shown here:

```
<project>
 <modelVersion>4.0.0</modelVersion>

 <name>A Well-Grounded Maven Plugin</name>
 <groupId>com.wellgrounded</groupId>
 <artifactId>wellgrounded-maven-plugin</artifactId>
 <packaging>maven-plugin</packaging>                    (1)
 <version>1.0-SNAPSHOT</version>                        (2)

 <properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>11</maven.compiler.source>
  <maven.compiler.target>11</maven.compiler.target>
 </properties>

 <dependencies>                                         (3)
  <dependency>
   <groupId>org.apache.maven</groupId>
   <artifactId>maven-plugin-api</artifactId>
   <version>3.0</version>
  </dependency>

  <dependency>
   <groupId>org.apache.maven.plugin-tools</groupId>
   <artifactId>maven-plugin-annotations</artifactId>
   <version>3.4</version>
   <scope>provided</scope>
  </dependency>
 </dependencies>
</project>
```

(1) Lets Maven know we intend to build a plugin package

(2) -SNAPSHOT is a typical suffix added to not-yet-released versions of a library. This shows up when pulling in the library because you must specify the full string 1.0-SNAPSHOT, for example, when asking for the dependency.

(3) Maven API dependencies our implementation will need

That gets us set to start adding code. Placing a Java file in the standard layout location, we implement what is called a `Mojo`—effectively a Maven *goal*, as follows:

```
package com.wellgrounded;

import org.apache.maven.plugin.AbstractMojo;
import org.apache.maven.plugin.MojoExecutionException;
import org.apache.maven.plugins.annotations.Mojo;

@Mojo(name = "wellgrounded")
public class WellGroundedMojo extends AbstractMojo
{
   public void execute() throws MojoExecutionException
   {
      getLog().info("Extending Maven for fun and profit.");
   }
}
```

Our class extends `AbstractMojo` and tells Maven via the `@Mojo` annotation what our goal name is. The body of the method takes care of whatever job we want. In this case, we simply log some text, but you have the full Java language and ecosystem available at this point to implement your goal.

To test the plugin in another project, we need to `mvn install` it, which will place our JAR into the local caching repository. Once there, we can pull our plugin into another project just like all the other "real" plugins we've seen already in this chapter, as follows:

```
<build>
 <plugins>
  <plugin>
   <groupId>com.wellgrounded</groupId>                    1
   <artifactId>wellgrounded-maven-plugin</artifactId>
   <version>1.0-SNAPSHOT</version>
   <executions>
    <execution>                                           2
     <phase>compile</phase>
     <goals>
      <goal>wellgrounded</goal>
     </goals>
    </execution>
   </executions>
  </plugin>
```

```
    </plugins>
  </build>
```

❶ References to our plugin coordinates by groupId and artifactId

❷ Binds our goal to the compile phase

With this in place, we can see our plugin in action when we compile, as shown here:

```
~: mvn compile
  [INFO] Scanning for projects...
  [INFO]
  [INFO] ------------------< com.wellgrounded:example >--------------
  [INFO] Building example 1.0-SNAPSHOT
  [INFO] --------------------------[ jar ]-------------------------
  [INFO]
  [INFO] - maven-resources-plugin:2.6:resources (default-resources) -
  [INFO] Using 'UTF-8' encoding to copy filtered resources.
  [INFO] skip non existing resourceDirectory /src/main/resources
  [INFO]
  [INFO] --- maven-compiler-plugin:3.1:compile (default-compile)  ---
  [INFO] Nothing to compile - all classes are up to date
  [INFO]
  [INFO] --- wellgrounded-maven-plugin:1.0-SNAPSHOT:wellgrounded  ---
  [INFO] Extending Maven for fun and profit.                    ❶
  [INFO] ------------------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------------------
  [INFO] Total time:  0.872 s
  [INFO] Finished at: 2020-08-16T22:26:20+02:00
  [INFO] ------------------------------------------------------------
```

❶ Our plugin running as part of the compile phase

It's worth noting that if we simply include the plugin without the `<executions>` element, we won't see our plugin show up anywhere in our project. Custom plugins must declare their desired phase in the lifecycle via the `pom.xml` file.

Visibility into the lifecycle and what goals are bound to what phases can be difficult, but fortunately there's a plugin to help with that. `buildplan-maven-plugin` brings clarity to your current tasks.

Although it can be included in a `pom.xml` like any other plugin, a useful alternative to avoid repetition is putting it in your user's `~/.m2/settings.xml` file, as shown next. `settings.xml` files are similar to `pom.xml` files in Maven, but they are not associated to any specific project:

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0
             https://maven.apache.org/xsd/settings-1.0.0.xsd">
 <pluginGroups>
  <pluginGroup>fr.jcgay.maven.plugins</pluginGroup>
 </pluginGroups>
</settings>
```

Once there, you can invoke it in any project building with Maven like this:

```
~: mvn buildplan:list

 [INFO] Scanning for projects...
 [INFO]
 [INFO] --------------------< com.wellgrounded:example >-----------------
 [INFO] Building example 1.0-SNAPSHOT
 [INFO] ----------------------------[ jar ]---------------------------
 [INFO]
 [INFO] ---- buildplan-maven-plugin:1.3:list (default-cli) @ example ----
 [INFO] Build Plan for example:
 ------------------------------------------------------------------------
 PLUGIN             | PHASE         | ID              | GOAL
 ------------------------------------------------------------------------
 jacoco-maven-plugin  | initialize      | default           | prep-agent
 maven-compiler-plugin| compile         | default-compile    | compile
 maven-compiler-plugin| test-compile    | default-testCompile| testCompile
 maven-surefire-plugin| test            | default-test       | test
 jacoco-maven-plugin  | test            | report             | report
 maven-jar-plugin     | package         | default-jar        | jar
 maven-failsafe-plugin| integration-test| default            | int-test
 maven-failsafe-plugin| verify          | default            | verify
 maven-install-plugin | install         | default-install    | install
 maven-deploy-plugin  | deploy          | default-deploy     | deploy
 [INFO] ------------------------------------------------------------------
 [INFO] BUILD SUCCESS
 [INFO] ------------------------------------------------------------------
 [INFO] Total time:  0.461 s
```

```
[INFO] Finished at: 2020-08-30T15:54:30+02:00
[INFO] ------------------------------------------------------------
```

NOTE If you don't want to add a plugin to your `pom.xml` *or* your `settings.xml`, you can just ask Maven to a run a command using the fully qualified plugin name! In our previous example, we can just say `mvn fr.jcgay.maven .plugins:buildplan-maven-plugin:list` and Maven will download the plugin and run it once. This is great for uncommon tasks or experimentation. Maven's documentation for authoring plugins (see **http://mng.bz/v6dx**) is thorough and well maintained, so do take a look when starting to implement your own plugins.

Maven remains among the most common build tools for Java and has been hugely influential. However, not everyone loves its strongly opinionated stance. Gradle is the most popular alternative, so let's see how it tackles the same problem space.

11.3 Gradle

Gradle came onto the scene after Maven and is compatible with much of the dependency management infrastructure Maven pioneered. It supports the familiar standard directory layout and provides a default build lifecycle for JVM projects, but unlike Maven, all of these features are fully customizable.

Instead of XML, Gradle uses a declarative domain-specific language (DSL) on top of an actual programming language (either Kotlin or Groovy). This typically results in concise build logic for simple cases and a lot of flexibility when things get more complex.

Gradle also includes a number of performance features for avoiding unnecessary work and processing tasks incrementally. This often provides faster builds and higher scalability. Let's get our feet wet by seeing how to run Gradle commands.

.3.1 Installing Gradle

Gradle can be installed from its website (**https://gradle.org/install**). Recent versions rely on having only JVM version 8 or greater. Once installed, you can run it at the command line, and it will default to displaying help, as shown here:

```
~: gradle

> Task :help

Welcome to Gradle 7.3.3.
```

```
To run a build, run gradlew <task> ...

To see a list of available tasks, run gradlew tasks

To see more detail about a task, run gradlew help --task <task>

To see a list of command-line options, run gradlew --help

For more detail on using Gradle, see
  https://docs.gradle.org/7.3.3/userguide/command_line_interface.html

For troubleshooting, visit https://help.gradle.org

BUILD SUCCESSFUL in 606ms
1 actionable task: 1 executed
```

This makes it easy to get started, but having a single global Gradle version isn't ideal. It is common for a developer to build multiple different projects that could each have different versions of Gradle.

To handle this, Gradle introduces the idea of a *wrapper*. The `gradle wrapper` task will capture a specific version of Gradle locally into your project. This is then accessed via the `./gradlew` or `gradlew.bat` commands. It's considered good practice to use the `gradlew` wrappers to avoid version incompatibilities so you may find yourself rarely actually running `gradle` itself directly.

NOTE It's recommended that you include the `gradle` and `gradlew*` results of the wrapper in source control but exclude the local caching of `.gradle`.

With the wrappers committed, anyone downloading your project gets the properly versioned build tooling without any additional installs.

.3.2 Tasks

Gradle's key concept is the *task*. A task defines a piece of work that can be invoked. Tasks can depend on other tasks, be configured via scripting, and added through Gradle's plugin system. These resemble Maven's goals but are conceptually more like functions. They have well-defined inputs and outputs and can be composed and chained. Whereas Maven goals must be associated to a given phase of the build life-cycle, Gradle tasks may be invoked and used in whatever fashion is convenient for you.

Gradle provides excellent introspection features. Key among these is the `./gradlew tasks` meta-task, which lists currently available tasks in your project. Before you've even declared anything, running `tasks` will present the following task list:

```
~: ./gradlew tasks

> Task :tasks

------------------------------------------------------------
Tasks runnable from root project
------------------------------------------------------------

Build Setup tasks
-----------------
init - Initializes a new Gradle build.
wrapper - Generates Gradle wrapper files.

Help tasks
----------
buildEnvironment - Displays all buildscript dependencies in root project
components - Displays the components produced by root project.
dependencies - Displays all dependencies declared in root project.
dependencyInsight - Displays insight for dependency in root project
dependentComponents - Displays dependent components in root project
help - Displays a help message.
model - Displays the configuration model of root project. [incubating]
outgoingVariants - Displays the outgoing variants of root project.
projects - Displays the sub-projects of root project.
properties - Displays the properties of root project.
tasks - Displays the tasks runnable from root project.
```

Providing the `--dry-run` flag to any task will display the tasks Gradle would have run, without performing the actions. This is useful for understanding the flow of your build system or debugging misbehaving plugins or custom tasks.

.3.3 What's in a script?

The heart of a Gradle build is its *buildscript*. This is a key difference between Gradle and Maven—not only is the format different but the entire philosophy is different, too. Maven POM files are XML-based, whereas in Gradle, the buildscript is an executable script written in a programming language—what's often referred to as a *domain-specific language* or DSL. Modern versions of Gradle support both Groovy and Kotlin.

Gradle's DSL approach started out with Groovy. As we learned when we met it briefly in chapter 8, Groovy is a dynamic language on the JVM, and it fits nicely with the goal of flexibility and concise build scripting. Since Gradle 5.0, however, another option has been available: Kotlin, which we covered in detail in chapter 9.

NOTE Kotlin buildscripts use the extension `.gradle.kts` instead of `.gradle`.

This makes a lot of sense because Kotlin is now the dominant language for Android development, where Gradle is the platform's official build tool. Sharing the same language across all parts of your project can be a great simplifying factor.

For our purposes, Kotlin is also more like Java than Groovy. Narrowing this language gap means that if you're new to the Gradle ecosystem, it might make sense to write your buildscript with Kotlin if you are coding in Java.

Groovy remains a prominent and very viable option, but we're going to double down on our Kotlin experience and use it for all of our following examples. Anything we show in this chapter can be expressed similarly in a Groovy buildscript with identical Gradle behavior. The Gradle documentation shows both DSL for all its examples.

.3.4 Using plugins

Gradle uses plugins to define everything about the tasks we use. As we saw earlier, listing tasks in a blank Gradle project doesn't say anything about building, testing, or deploying. All of that comes from plugins.

Numerous plugins ship with Gradle itself, so using them requires only a declaration in your `build.gradle.kts`. A key one is the `base` plugin, shown here:

```
plugins {
  base
}
```

A look at our tasks after including the `base` plugin reveals some common build life-cycle tasks that we might expect, as shown next:

```
~:./gradlew tasks

 > Task :tasks
```

```
------------------------------------------------------------
Tasks runnable from root project
------------------------------------------------------------

Build tasks
-----------
assemble - Assembles the outputs of this project.
build - Assembles and tests this project.
clean - Deletes the build directory.

... Other tasks omitted for length

Verification tasks
------------------
check - Runs all checks.

...

BUILD SUCCESSFUL in 640ms
1 actionable task: 1 executed
```

With that in place, let's get building a Gradle project for our code.

.3.5 Building

Although Gradle allows for customization to your heart's content, it defaults to expecting the same code layout that Maven established and popularized. For many (perhaps even most) projects, it doesn't make sense to change this layout, although it is possible to do so.

Let's start with a basic Java library. To do this, we create the following source tree:

```
.
├── build.gradle.kts
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar                1
│       └── gradle-wrapper.properties         1
├── gradlew                                   1
├── gradlew.bat                               1
├── settings.gradle.kts
└── src
    └── main
```

```
    └── java
        └── com
            └── wellgrounded
                └── AwesomeLib.java
```

❶ These files were automatically created by the Gradle wrapper command.

The `base` plugin doesn't know anything about Java, so we need a plugin with more awareness. For our use case of a plain Java JAR, we'll use Gradle's `java-library` plugin, shown next. This plugin builds on all the necessary parts from `base`—in practice, you'll rarely see the `base` plugin alone in a Gradle build. That's because plugins can apply other plugins to build off of them, like composition in object-oriented programming:

```
plugins {
  `java-library`  ❶
}
```

❶ Backticks (not apostrophes) are used around plugin names when they include special characters such as - here.

This yields a growing set of tasks in our build section, as shown here:

```
Build tasks
  -----------
  assemble - Assembles the outputs of this project.
  build - Assembles and tests this project.
  buildDependents - Assembles and tests this project and dependent projects.
  buildNeeded - Assembles and tests this project and dependent projects.
  classes - Assembles main classes.
  clean - Deletes the build directory.
  jar - Assembles a jar archive containing the main classes.
  testClasses - Assembles test classes.
```

In Gradle's terminology, `assemble` is the task that will compile and package up a JAR file. A dry run shows all of the steps, some of which the default tasks list doesn't show:

```
./gradlew assemble --dry-run
  :compileJava SKIPPED
  :processResources SKIPPED
  :classes SKIPPED
```

```
:jar SKIPPED
:assemble SKIPPED
```

Running `./gradlew assemble` generates output in the `build` directory as follows:

```
.
└── build
    ├── classes
    │   └── java
    │       └── main
    │           └── com
    │               └── wellgrounded
    │                   └── Main.class
    └── libs
        └── wellgrounded.jar
```

Making an application

A plain JAR is a good start, but eventually you want to run an application. This takes more configuration, but again the pieces are available by default.

We'll change up our plugins and tell Gradle what the main class is for our application. We also can see several of the nice features that Kotlin brings in just this brief snippet:

```
plugins {                                   ❶
  application                               ❷
}

application {
  mainClass.set("wgjd.Main")
}

tasks.jar {                                 ❸
  manifest {
    attributes("Main-Class" to application.mainClass)    ❹
  }
}
```

❶ Kotlin's optional parentheses when the final argument is a lambda

❷ Plugin that knows how to compile and run a Java app

❸ Task for assembling a JAR with a modified manifest

❹ Kotlin uses to syntax to declare a hash map in place (aka a hash literal).

Building with `./gradlew build` gets us the same JAR output as before, but if we execute `java -jar build/libs/wellgrounded.jar`, our test program will run. Alternatively, the `application` plugin also supports `./gradlew run` to directly load and execute your main class for you.

NOTE The `application` plugin requires only that `mainClass` be set, but excluding the `tasks.jar` configuration will yield a JAR that `./gradlew run` knows how to start but `java -jar` doesn't. Definitely not recommended!

We now have the pieces we need to examine another key feature of Gradle: its ability to avoid work and reduce our build times.

.3.6 Work avoidance

To run builds as fast as possible, Gradle tries to avoid repeating unnecessary work. One strategy for this is incremental build. Every task in Gradle declares its inputs and outputs. Gradle uses this information to check whether anything has changed since the last time the build ran. If there is no change, Gradle skips the task and reuses its outputs from the previous build.

NOTE You shouldn't regularly run `clean` when using Gradle, because Gradle will ensure that the necessary—and only the necessary—work is done to produce the build results.

We can see this with our application build by taking a look at the build times after one full run (forced clean) and a second run, shown here:

```
~: ./gradlew clean build

BUILD SUCCESSFUL in 2s
13 actionable tasks: 13 executed

~: ./gradlew build

BUILD SUCCESSFUL in 804ms
12 actionable tasks: 12 up-to-date
```

Incremental build can only reuse outputs from the last execution of a task in the same location on this computer. Gradle does even better: the Build Cache allows reusing task outputs from any earlier build—or even a build run elsewhere.

This feature can be enabled in your project via a property with the `--build-cache` command-line flag. We can see that even the following `clean` build is faster because it can reuse cached outputs from the prior execution:

```
~: ./gradlew clean build --build-cache

 BUILD SUCCESSFUL in 2s
 13 actionable tasks: 13 executed

 ~: ./gradlew clean build --build-cache

 BUILD SUCCESSFUL in 1s
 13 actionable tasks: 6 executed, 7 from cache
```

Performance is a key feature of Gradle in keeping your project build times down even as the size of your code grows. Other abilities exist that we won't have time to cover, such as incremental Java compilation, the Gradle Daemon, and parallel task and test execution.

No person is an island. Similarly, few applications get far without pulling in other library dependencies. This is a major subject in Gradle and a point of considerable difference from Maven.

.3.7 Dependencies in Gradle

To start introducing dependencies, we must first tell Gradle which repositories it can download from. There are built-in functions for `mavenCentral` (shown next) and `google`. You can use more detailed APIs to configure other repositories, including your own private instances:

```
repositories {
  mavenCentral()
}
```

We can then introduce our dependencies via the standard coordinate format popularized by Maven. Much like Maven had the `<scope>` element to control where a given depen-

dency was used, Gradle expresses this through *dependency configurations*. Each configuration tracks a particular set of dependencies. Your plugins define which configurations are available, and you add to a configuration's list with a function call. For example, to pull the SLF4J library (http://www.slf4j.org/) to help with logging, we'd use the following configurations:

```
dependencies {
  implementation("org.slf4j:slf4j-api:1.7.30")
  runtimeOnly("org.slf4j:slf4j-simple:1.7.30")
}
```

In this example, our code directly calls classes and methods in `slf4j-api`, so it is included via the `implementation` configuration. This makes it available during compilation and running the application. Our application should never directly call methods in `slf4j-simple`, though—that's done strictly through the `slf4j-api`—so requesting `slf4j-simple` as `runtimeOnly` ensures that code isn't available during compilation, preventing us from misusing the library. This achieves the same purpose as the `<scope>` element with dependencies in Maven.

The distinction between dependencies we use directly and those just needed in our classpath at runtime isn't the only way to distinguish differences between dependencies. For library authors in particular, there is also a distinction between libraries that we use and those that are part of our public API. If a dependency is part of the public API of a project, we can mark it with `api`. In the following example, we're declaring that Guava is part of the public API of our project:

```
dependencies {
  api("com.google.guava:guava:31.0.1-jre")
}
```

Configurations can extend one another, much like deriving from a base class. Gradle applies this feature in many areas. For example when creating classpaths, Gradle uses a `compileClasspath` and `runtimeClassPath`, which extends `implementation` and `runtimeOnly`. You aren't meant to directly add to the `*Classpath` configurations—the dependencies we add to their base configurations build up the resulting classpath configuration, as shown in figure 11.6.

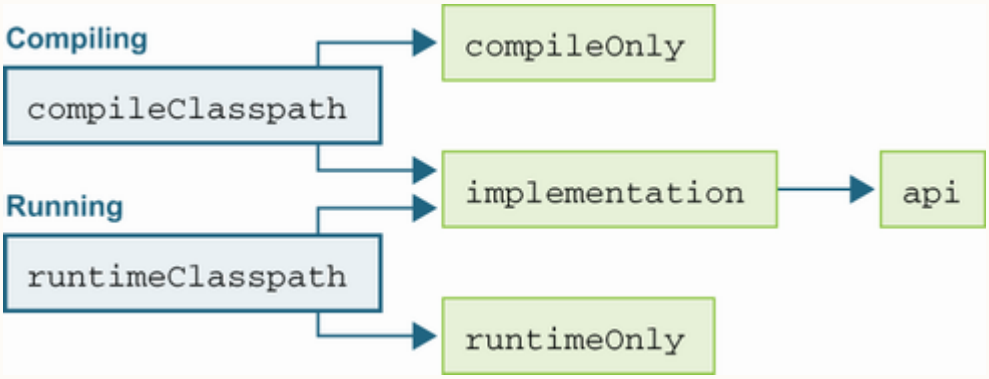


Figure 11.6 Hierarchy of Gradle configurations

Table 11.1 shows some primary configurations available when using the Java plugin that ships with Gradle, along with an indication of what other configurations each extends from. A full list is available in the Java plugin documentation at **http://mng.bz/445B**.

NOTE Version 7 of Gradle removed a number of long-standing deprecated configurations, for instance, `compile` and `runtime`. If you're reading around the internet, you may still find reference to these but should move to the newer options, `implementation` (or `api`) and `runtimeOnly`.

Table 11.1 Typical Gradle dependency configurations

Name	Purpose	Extends
`api`	Primary dependencies that are part of the project's external, public API	
implementation	Primary dependencies used during compiling and running	
`compileOnly`	Dependencies needed only during compilation	
compileClasspath	Configuration Gradle uses to look up compilation classpath	compileOnly , implementation
`runtimeOnly`	Dependencies needed only during runtime	
runtimeClasspath	Configuration Gradle uses to look up runtime classpath	runtimeOnly , implementation
`testImplementation`	Dependencies used during compiling and running tests	`implementation`
testCompileOnly	Dependencies needed only during compilation of tests	
`testCompileClasspath`	Configuration Gradle uses to look up test compilation classpath	`testCompileOnly` , `testImplementation`
testRuntimeOnly	Dependencies needed only during runtime	runtimeOnly
`testRuntimeClasspath`	Configuration Gradle uses to look up test runtime classpath	`testRuntimeOnly` , `testImplementation`

Name	Purpose	Extends
archives	List of output JARs from our project	

Like Maven, Gradle uses the package information to create a transitive dependency tree. However, Gradle's default algorithm for handling version conflicts differs from Maven's "closest-to-the-root-wins" approach. When resolving, Gradle walks the full dependency tree to determine all the requested versions for any given package. From the full set of requested versions, Gradle will then default to the *highest* available version.

This approach avoids some unexpected behavior in Maven's approach—for instance, changes in the ordering/depth of packages could result in different resolution. Gradle can also use additional information such as rich version constraints to customize the resolution process. Even further, if Gradle can't satisfy the defined constraints, it will fail the build with a clear message rather than choose a version that may be problematic.

Given this, Gradle provides rich APIs to override and control its resolution behavior. It also has solid introspecting tools built in to pull back the curtains when something goes wrong. A key command when transitive dependency problems rear their ugly head is `./gradlew dependencies`, shown here:

```
~: ./gradlew dependencies

testImplementation - Implementation only dependencies for compilation 'test'
\--- org.junit.jupiter:junit-jupiter-api:5.8.1 (n)

... Other configurations skipped for length

testRuntimeClasspath - Runtime classpath of compilation 'test'
+--- org.junit.jupiter:junit-jupiter-api:5.8.1
|    +--- org.junit:junit-bom:5.8.1
|    |    +--- org.junit.jupiter:junit-jupiter-api:5.8.1 (c)
|    |    +--- org.junit.jupiter:junit-jupiter-engine:5.8.1 (c)
|    |    +--- org.junit.platform:junit-platform-commons:1.8.1 (c)
|    |    \--- org.junit.platform:junit-platform-engine:1.8.1 (c)
|    +--- org.opentest4j:opentest4j:1.2.0
|    \--- org.junit.platform:junit-platform-commons:1.8.1
|         \--- org.junit:junit-bom:5.8.1 (*)
\--- org.junit.jupiter:junit-jupiter-engine:5.8.1
     +--- org.junit:junit-bom:5.8.1 (*)
     +--- org.junit.platform:junit-platform-engine:1.8.1
```

```
|    +--- org.junit:junit-bom:5.8.1 (*)
|    +--- org.opentest4j:opentest4j:1.2.0
|    \--- org.junit.platform:junit-platform-commons:1.8.1 (*)
\--- org.junit.jupiter:junit-jupiter-api:5.8.1 (*)

testRuntimeOnly - Runtime only dependencies for compilation 'test'
\--- org.junit.jupiter:junit-jupiter-engine:5.8.1 (n)
```

In a large project, this output can be overwhelming, so `dependencyInsight` lets you focus on the specific dependency you care about like this:

```
~: ./gradlew dependencyInsight \
      --configuration testRuntimeClasspath \
      --dependency junit-jupiter-api

> Task :dependencyInsight
org.junit.jupiter:junit-jupiter-api:5.8.1 (by constraint)
  variant "runtimeElements" [
    org.gradle.category               = library
    org.gradle.dependency.bundling    = external
    org.gradle.jvm.version            = 8 (compatible with: 11)
    org.gradle.libraryelements        = jar
    org.gradle.usage                  = java-runtime
    org.jetbrains.kotlin.localToProject = public (not requested)
    org.jetbrains.kotlin.platform.type  = jvm
    org.gradle.status                 = release (not requested)
  ]

org.junit.jupiter:junit-jupiter-api:5.8.1
+--- testRuntimeClasspath
+--- org.junit:junit-bom:5.8.1
|    +--- org.junit.platform:junit-platform-engine:1.8.1
|    |    +--- org.junit:junit-bom:5.8.1 (*)
|    |    \--- org.junit.jupiter:junit-jupiter-engine:5.8.1
|    |         +--- testRuntimeClasspath
|    |         \--- org.junit:junit-bom:5.8.1 (*)
|    +--- org.junit.platform:junit-platform-commons:1.8.1
|    |    +--- org.junit.platform:junit-platform-engine:1.8.1 (*)
|    |    +--- org.junit:junit-bom:5.8.1 (*)
|    |    \--- org.junit.jupiter:junit-jupiter-api:5.8.1 (*)
|    +--- org.junit.jupiter:junit-jupiter-engine:5.8.1 (*)
|    \--- org.junit.jupiter:junit-jupiter-api:5.8.1 (*)
\--- org.junit.jupiter:junit-jupiter-engine:5.8.1 (*)
```

```
(*) - dependencies omitted (listed previously)
```

Dependency conflicts can be hard to resolve. The best approach, if possible, is to use the dependency tools in Gradle to find mismatches and upgrade to mutually compatible versions. Ah, to live in a world where that were always possible!

Let's revisit the example we had previously where two versions of an internal helper library were bringing in `assertj` at incompatible major versions. In that case, `first-test-helper` was dependent on `assertj-core` 3.21.0, whereas `second-test-helper` wanted 2.9.1.

Gradle's `constraints` provides a mechanism to inform the resolution process how we'd like it to choose versions, as shown here:

```
dependencies {
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.1")

  testImplementation(
     "com.wellgrounded:first-test-helper:1.0.0")    ❶
  testImplementation(
     "com.wellgrounded:second-test-helper:2.0.0")   ❶

  constraints {
    testImplementation(
      "org.assertj:assertj-core:3.1.0") {           ❷
       because("Newer incompatible because...")     ❸
    }
  }
}
```

❶ All dependencies just ask for what they want as before.

❷ Gradle will respect this constraint or fail the resolution.

❸ It's good practice to use because for documenting why we're intervening because Gradle's tooling can use that text, versus comments in the script, which are useful only to human readers.

If you really need to get precise, you can set a version using `strictly`, which will override any other resolution, as follows:

```
dependencies {
 testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.1")
 testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.1")

 testImplementation(
  "com.wellgrounded:first-test-helper:1.0.0")      (1)
 testImplementation(
  "com.wellgrounded:second-test-helper:2.0.0")     (1)

 testImplementation("org.assertj:assertj-core") {
  version {
   strictly("3.1.0")                    (2)
  }
 }
}
```

(1) All dependencies just ask for what they want as before.

(2) Forces version 3.1.0. This won't match 3.1 or any other related version.

If these mechanisms aren't enough or a library simply has an error in its listed dependencies, you can also ask Gradle to just ignore a given group or artifact via `exclude` as follows:

```
dependencies {
 testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.1")
 testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.1")

 testImplementation(
  "com.wellgrounded:first-test-helper:1.0.0")      (1)
 testImplementation(
  "com.wellgrounded:second-test-helper:2.0.0") {   (2)
  exclude(group = "org.assertj")
 }
}
```

(1) Dependency from first-test-helper will be chosen.

(2) Gradle will ignore the org.assertj dependencies from the second helper.

This is a more drastic option, though, and as written here applies only to the dependency that we apply the `exclude` to. If we can find a solution using `constraints`, we'll be better off in the long run.

As we've mentioned in prior sections, manually forcing dependency versions is a last resort and deserves special attention to ensure you aren't getting runtime exceptions. A robust test suite can be critical to save time when ensuring your mix of libraries works together smoothly.

.3.8 Adding Kotlin

As we've discussed in both chapter 8 and the Maven section of this chapter, the ability to add another language to a project is a huge benefit of running on the JVM.

Adding Kotlin shows off the benefits of Gradle's scripted approach over Maven's more static XML-based configuration. Following the standard multilingual layout from our original code yields this:

```
.
├── build.gradle.kts
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle.kts
└── src
    ├── main
    │   ├── java
    │   │   └── com
    │   │       └── wellgrounded
    │   │           └── Main.java
    │   └── kotlin                    (1)
    │       └── com
    │           └── wellgrounded
    │               └── kotlin
    │                   └── MessageFromKotlin.kt
    └── test
        └── java
            └── com
                └── wellgrounded
                    └── MainTest.java
```

❶ Our additional Kotlin code appears under the kotlin subdirectories.

We enable Kotlin support via a Gradle plugin in our `build.gradle.kts` like this:

```
plugins {
  application
  id("org.jetbrains.kotlin.jvm") version "1.6.10"
}
```

And that's it. Because of Gradle's flexibility, the plugin is able to alter the build order and add the necessary `kotlin-stdlib` dependencies without us having to take additional steps.

.3.9 Testing

The `assemble` task we first discussed will compile and package your main code, but we need to compile and run our tests as well. The `build` task is configured by default for just that, as shown here:

```
./gradlew build --dry-run
  :compileJava SKIPPED
  :processResources SKIPPED
  :classes SKIPPED
  :jar SKIPPED
  :assemble SKIPPED
  :compileTestJava SKIPPED
  :processTestResources SKIPPED
  :testClasses SKIPPED
  :test SKIPPED
  :check SKIPPED
  :build SKIPPED
```

We'll add a test case using the standard locations as follows:

```
src
  └── test
      └── java
          └── com
              └── wellgrounded
                  └── MainTest.java
```

Next up, we need to add our test framework to the right dependency configuration to make it available to our code. We also let Gradle know that it should use JUnit when running the test tasks, as shown next:

```
dependencies {
  ....
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.1")
}

tasks.named<Test>("test") {
  useJUnitPlatform()
}
```

The `testImplementation` configuration makes `org.junit.jupiter` available when building and executing test—but not main—code. When we next run the `./gradlew build`, you'll see that it's downloading the library into our local cache if it wasn't already there.

Full listings with stack traces, including an HTML-based report, are generated under `build/reports/test`.

.3.10 Automating static analysis

The build is a great place to add functionality to protect your project. One type of check beyond unit testing is static analysis. There are several tools in this category, but SpotBugs (**https://spotbugs.github.io/**) (the successor to FindBugs) is an easy one to get started with. Note that most of these tools have plugins for Maven as well as Gradle, so the treatment shown here is just to give you a taste of the possibilities:

```
plugins {
  application
  id("com.github.spotbugs") version "4.3.0"
}
```

If we deliberately introduce a problem in our code (e.g., implementing `equals` on a class without also overriding `hashCode`), a typical `./gradlew check` will let us know there's a problem, as illustrated here:

```
~:./gradlew check
```

```
> Task :spotbugsTest FAILED

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':spotbugsTest'.
> A failure occurred while executing SpotBugsRunnerForWorker
  > Verification failed: SpotBugs violation found:
    2. SpotBugs report can be found in build/reports/spotbugs/test.xml

* Try:
Run with --stacktrace option to get the stack trace.
Run with --info or --debug option to get more log output.
Run with --scan to get full insights.

* Get more help at https://help.gradle.org

BUILD FAILED in 1s
5 actionable tasks: 3 executed, 2 up-to-date
```

As with unit testing failures, report files are under `build/reports/spotbugs`. Out of the box, SpotBugs may generate only an XML file, which, although nice for computers, is less useful to most people. We can configure the plugin to emit HTML for us as follows:

```
tasks.withType<com.github.spotbugs.snom.SpotBugsTask>()    ❶
 .configureEach {                                          ❷
  reports.create("html") {                                 ❸
    isEnabled = true
    setStylesheet("fancy-hist.xsl")
  }
}
```

❶ tasks.withType looks up tasks for us in a typesafe manner.

❷ configureEach runs the block as if we had written tasks.spotbugsMain { } and then tasks.spotbugsTest { } with the same code.

❸ The remaining configuration is taken from the project's README on GitHub (**http://mng.bz/Qvdm**).

.3.11 Moving beyond Java 8

In chapter 1, we noted the following series of modules that belonged with Java Enterprise Edition but were present in the core JDK. These were deprecated with JDK 9 and removed with JDK 11 but remain available as external libraries:

- `java.activation` (JAF)
- `java.corba` (CORBA)
- `java.transaction` (JTA)
- `java.xml.bind` (JAXB)
- `java.xml.ws` (JAX-WS, plus some related technologies)
- `java.xml.ws.annotation` (Common Annotations)

If your project relies on any of these modules, your build might break when you move to a more recent JDK. Fortunately, you can add the following simple dependencies in your `build.gradle.kts` to address the issue:

```
dependencies {
  implementation("com.sun.activation:jakarta.activation:1.2.2")
  implementation("org.glassfish.corba:glassfish-corba-omgapi:4.2.1")
  implementation("javax.transaction:javax.transaction-api:1.3")
  implementation("jakarta.xml.bind:jakarta.xml.bind-api:2.3.3")
  implementation("jakarta.xml.ws:jakarta.xml.ws-api:2.3.3")
  implementation("jakarta.annotation:jakarta.annotation-api:1.3.5")
}
```

.3.12 Using Gradle with modules

Like Maven, Gradle supports the JDK module system fully. Let's break down what we need to alter to use our modular projects with Gradle.

A MODULAR LIBRARY

A modular library typically has two major structural differences: the change from using main to the module name in the directory under `src`, and the addition of a `module-info.java` file at the root of our module, as shown next:

```
.
├── build.gradle.kts
├── gradle
│   └── wrapper
```

```
│           ├── gradle-wrapper.jar
│           └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle.kts
└── src
    └── com.wellgrounded.modlib                ❶
        └── java
            ├── com
            │   wellgrounded
            │       ├── hidden                 ❷
            │       │       └── CantTouchThis.java
            │       └── visible                ❸
            │               └── UseThis.java
            └── module-info.java               ❹
```

❶ The directory name aligned with our module

❷ We intend to keep this package hidden.

❸ This package will be exported for use outside this module.

❹ The module-info.java declarations for this module

Gradle doesn't automatically find our altered source location, so we need to give it a hint in `build.gradle.kts` where to look as follows:

```
sourceSets {
  main {
    java {
      setSrcDirs(listOf("src/com.wellgrounded.modlib/java"))
    }
  }
}
```

The `module-info.java` file contains the typical declarations that we saw demonstrated earlier in this chapter and in chapter 2. We'll name our module and select one, but not both, of our packages to export like this:

```
module com.wellgrounded.modlib {
   exports com.wellgrounded.modlib.visible;
}
```

That's all that's required to make our library consumable as a module. Next we'll use the library from a modular app.

A MODULAR APPLICATION

When we set out to test our modular application under Maven, the simplest way to share the library we'd created with our app was to install it to the local Maven repository. This is also supported from Gradle via the `maven-publish` plugin, but we have another option that is worth understanding the mechanics of.

Our modular application has a standard layout as shown next. For ease of testing, we'll make sure the top-level directories live next to each other:

```
mod-lib                                    (1)
└── ...

mod-app
├── build.gradle.kts
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle.kts
└── src
    └── com.wellgrounded.modapp            (2)
        └── java
            ├── com
            │   └── wellgrounded
            │       └── Main.java
            └── module-info.java           (3)
```

(1) The mod-lib library source is at the same level as our mod-app application.

(2) The directory name is aligned with the module name.

(3) We use module-info.java to declare this a modularized application.

Our `module-info.java` file tells our name and module requirements, as shown here:

```
module com.wellgrounded.modapp {      ❶
   requires com.wellgrounded.modlib;  ❷
}
```

❶ Our module name

❷ Our requirement on our library's exported packages

For testing our local library, rather than installing it, we'll refer to it locally for the moment, as shown in the next code snippet. This can be accomplished using the `files` function in the spot where we'd previously have given the GAV coordinates for our dependency. This obviously won't work once we're ready to start sharing and deploying, but it's a quick move to get our local testing started:

```
dependencies {
  implementation(files("../mod-lib/build/libs/gradle-mod-lib.jar"))
}
```

Next up, current versions of Gradle require a hint that we want it to sniff out which dependencies are modular to properly put them on the module path instead of the classpath as follows. This may become a default eventually, but at the time of this writing (Gradle 7.3) it remains an opt-in:

```
java {
  modularity.inferModulePath.set(true)
}
```

Last and most mundanely, like our library, we need to let Gradle know about our non-Maven standard file location as follows:

```
sourceSets {
  main {
    java {
      setSrcDirs(listOf("src/com.wellgrounded.modapp/java"))
    }
  }
}
```

With all this in place, `./gradlew build run` has the expected result. If we attempt to use a package from the library that isn't exported, we confront the error at compilation time as shown here:

```
> Task :compileJava FAILED
/mod-app/src/com.wellgrounded.modapp/java/com/wellgrounded/Main.java:4:
error: package com.wellgrounded.modlib.hidden is not visible

import com.wellgrounded.modlib.hidden.CantTouchThis;
                    ^
  (package com.wellgrounded.modlib.hidden is declared in module
   com.wellgrounded.modlib, which does not export it)
1 error
```

JLink

A capability we saw in chapter 2 that modules unlock is the ability to create a streamlined environment for an application to work in, with only the dependencies it requires. This is possible because the module system gives us concrete guarantees about which modules our code uses, so tooling can construct the necessary, minimal set of modules.

NOTE JLink can work only with fully modularized applications. If an application is still loading some code via the classpath, JLink can't succeed in making a safe, complete image.

This feature is most evident through the `jlink` tool. For a modular application, JLink can produce a fully functioning JVM image that can be run without depending on a system-installed JVM.

Let's revisit the application from chapter 2 that we demonstrated JLink with to see how Gradle plugins streamline the management. The sample application, available in the supplement, uses JDK classes to attach to all the running JVM processes on a machine and display various information about them.

In the modular application we're going to package, an important bit to review is the application's own `module-info.java` declarations. As shown next, these tell us what JLink will need to pull into it's custom image for our build to work:

```
module wgjd.discovery {
  exports wgjd.discovery;
```

```
  requires java.instrument;
  requires java.logging;
  requires jdk.attach;
  requires jdk.internal.jvmstat;   (1)
}
```

(1) Red flag: note the jdk.internal package that we're reaching into!

Before we even get started with JLink, moving from hand-compiling to our Gradle build takes a little extra configuration. We need to apply the same modularization changes explained in the preceding section as a start as follows. But even once those are in place, we can't compile successfully:

```
~:./gradlew build

> Task :compileJava FAILED
/gradle-jlink/src/wgjd.discovery/wgjd/discovery/VMIntrospector.java:4:
error: package sun.jvmstat.monitor is not visible
  import sun.jvmstat.monitor.MonitorException;
                ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
   which does not export it to module wgjd.discovery)

/gradle-jlink/src/wgjd.discovery/wgjd/discovery/VMIntrospector.java:5:
error: package sun.jvmstat.monitor is not visible
  import sun.jvmstat.monitor.MonitoredHost;
                ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
   which does not export it to module wgjd.discovery)

/gradle-jlink/src/wgjd.discovery/wgjd/discovery/VMIntrospector.java:6:
error: package sun.jvmstat.monitor is not visible
  import sun.jvmstat.monitor.MonitoredVmUtil;
                ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
   which does not export it to module wgjd.discovery)

/gradle-jlink/src/wgjd.discovery/wgjd/discovery/VMIntrospector.java:7:
error: package sun.jvmstat.monitor is not visible
  import sun.jvmstat.monitor.VmIdentifier;
                ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
   which does not export it to module wgjd.discovery)
```

```
4 errors

FAILURE: Build failed with an exception.
```

The module system is letting us know that we're breaking the rules by trying to use classes that are in `jdk.internal.jvmstat`. Our module, `wgjd.discovery` is not included in the `jdk.internal.jvmstat` list of allowed modules. Understanding the rules and the risks we're taking, we can use `--add-exports` to force our module into the list. This is done via a compiler flag, and looks like this in our Gradle configuration:

```
tasks.withType<JavaCompile> {
 options.compilerArgs = listOf(
    "--add-exports",
    "jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery")
}
```

With that we get a clean compile and we can turn to using JLink to package it up. The plugin with the most mindshare today is `org.beryx.jlink`, known in the documentation as "The Badass JLink Plugin" (**https://badass-jlink-plugin.beryx.org**). We add it to our Gradle project with a plugin line.

```
plugins {
 id("org.beryx.jlink") version("2.23.3")  ❶
}
```

❶ This plugin automatically applies application for us, so we don't need to repeat that declaration.

After adding that, we'll see a `jlink` task in our list, which we can run straight away. The result will show up in the `build/image` directory like this:

```
build/image/
├── bin
│   ├── gradle-jlink
│   ├── gradle-jlink.bat
│   ├── java
│   └── keytool
├── conf
│   └── ... various configuration files
├── include
```

```
│   └── ... require headers
├── legal
│   └── ... license and legal information for all included modules
├── lib
│   └── ... library files and dependencies for our image
└── release
```

The `build/image/bin/java` is our custom JVM with only our application's module dependencies available to it. You can run it just like you would your normal `java` command from the terminal as follows:

```
~:build/image/bin/java -version
openjdk version "11.0.6" 2020-01-14
OpenJDK Runtime Environment AdoptOpenJDK (build 11.0.6+10)
OpenJDK 64-Bit Server VM AdoptOpenJDK (build 11.0.6+10, mixed mode)
```

We can pass `build/image/bin/java` our module to start up, but the plugin has neatly generated a startup script at `build/image/bin/gradle-jlink` (named after our project and shown next) that we can use instead. But not all is well with our newly minted image:

```
~:build/image/bin/gradle-jlink

Java processes:
PID    Display Name    VM Version    Attachable
Exception in thread "main" java.lang.IllegalAccessError:
 class wgjd.discovery.VMIntrospector (in module wgjd.discovery) cannot
   access class sun.jvmstat.monitor.MonitorException (in module
   jdk.internal.jvmstat) because module jdk.internal.jvmstat does not
   export sun.jvmstat.monitor to module wgjd.discovery
 wgjd.discovery/wgjd.discovery.VMIntrospector.accept(VMIntrospector.java:19)
 wgjd.discovery/wgjd.discovery.Discovery.main(Discovery.java:26)
```

This isn't an entirely unfamiliar error message—it's another flavor of the same access issue we solved with the compiler options earlier. Apparently we need to inform the application startup of our module-cheating needs as well. Fortunately, the plugin has extensive configuration for the parameters both to run `jlink` and for the resulting scripts created for us, as shown here:

```
jlink {
  launcher{
```

```
  jvmArgs = listOf(
          "--add-exports",
          "jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery")
 }
}
```

With that addition, the startup script gets everything running as follows:

```
~:build/image/bin/gradle-jlink
Java processes:
PID   Display Name   VM Version   Attachable
833 wgjd.discovery/wgjd.discovery.Discovery   11.0.6+10   true
276 org.jetbrains.jps.cmdline.Launcher /Applications/IntelliJ IDEA CE.app...
```

It's worth noting that the image we generated here defaults to targeting the same operating system that JLink is running on, as illustrated in the next code sample. However, that isn't required—cross-platform support is available. The primary requirement is that you have the files from the target platform's JDK installation available. These are easily available from sources such as the Eclipse Adoptium website at **https://adoptium.net/**:

```
jlink {
 targetPlatform("local",
          System.getProperty("java.home"))    ❶
 targetPlatform("linux-x64",
          "/linux_jdk-11.0.10+9")          ❷

 launcher{
  jvmArgs = listOf(
          "--add-exports",
          "jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery")
 }
}
```

❶ Builds an image based on whatever the local JDK is

❷ Builds an image pointed to a Linux JDK we've downloaded

Once you start targeting specific platforms, the plugin will put additional directories in the `build/image` results. Obviously, you'll have to take those results to a matching system to test them.

A final roadblock that may come up in trying to use JLink is its restrictions around automatically named modules. Although the feature to just add a name into the JAR manifest and get some basic ability to participate in the modular world is great for migrations, JLink sadly doesn't support it.

The Badass JLink Plugin, though, has you covered. It will repackage any automatically named modules into a proper module that JLink can consume. The documentation (found at **http://mng.bz/XZ2Y**) gives full coverage to this feature, which may be the difference between JLink working or not, depending on your application's dependencies.

.3.13 Customizing

One of Gradle's biggest strengths is its open-ended flexibility. Without pulling in plugins, it doesn't even have a concept of a build lifecycle. You can add tasks and reconfigure existing tasks with few restrictions. There's no need to keep a `scripts` directory around in your project with random tooling—your custom needs can be integrated right into your day-to-day build and testing tool.

Custom tasks

Defining a custom task can be done directly in your `build.gradle.kts` file like this:

```
tasks.register("wellgrounded") {
  println("configuring")
  doLast {
    println("Hello from Gradle")
  }
}
```

Running this will produce the following output:

```
~: ./gradlew wellgrounded
configuring...

> Task :wellgrounded
Hello from Gradle
```

The `println("configuring")` line is run during setup of the task, but the contents of the `doLast` block happens when the task actually ran. We can confirm this by doing a dry-run on our task as follows:

```
~: ./gradlew wellgrounded --dry-run
 configuring...
 :wellgrounded SKIPPED
```

Tasks can be configured to depend on other tasks, as shown next:

```
tasks.register("wellgrounded") {
  println("configuring...")
  dependsOn("assemble")
  doLast {
    println("Hello from Gradle")
  }
}
```

This technique applies equally well to tasks you didn't author—you can look them up and add your task as a dependency like so:

```
tasks {
  named<Task>("help") {
    dependsOn("wellgrounded")
  }
}
~: ./gradlew help
configuring...

> Task :wellgrounded
Hello from Gradle

> Task :help

Welcome to Gradle 7.3.3.

To run a build, run gradlew <task> ...

To see a list of available tasks, run gradlew tasks

To see more detail about a task, run gradlew help --task <task>

To see a list of command-line options, run gradlew --help

For more detail on using Gradle, see
  https://docs.gradle.org/7.3.3/userguide/command_line_interface.html
```

```
For troubleshooting, visit https://help.gradle.org
```

It's extremely powerful to be able to write custom tasks directly in your build file. However, putting them in `build.gradle.kts` has a few rather severe limitations: they cannot be easily shared between projects, and they aren't easy to write automated tests against. Gradle plugins are built to address just those issues.

Creating custom plugins

Gradle plugins are implemented as JVM code. They can be provided directly in your project as source files, or they can be pulled in through libraries. Many plugins have been written in Groovy, the original scripting language supported by Gradle, but you can do it in any JVM language. For the largest compatibility and to minimize issues with specific language idioms, if you plan to share your plugin, writing it in Java is a good idea.

Plugins can be coded directly in your buildscript, and we'll demonstrate the main APIs using that technique. When you're ready to share, you can pull the code into its own separate project. Here is an equivalent to our earlier `wellgrounded` task:

```
class WellgroundedPlugin : Plugin<Project> {   ❶
  override fun apply(project: Project) {
    project.task("wellgrounded") {          ❷
      doLast {
        println("Hello from Gradle")
      }
    }
  }
}

apply<WellgroundedPlugin>()                 ❸
```

❶ Derives from Plugin

❷ Uses familiar project-level API and task implementation

❸ Use apply to actually use the plugin—it isn't automatically invoked like our earlier task definition.

Apart from sharing, authoring tasks as plugins allows us more ability to customize configuration. The standard Gradle object representing our `Project` has a specific place

where plugin configurations live under an `extensions` property. We can add to these extensions with our own `Extension` objects as follows:

```
open class WellgroundedExtensions {
  var count: Int = 1
}

class WellgroundedPlugin : Plugin<Project> {
  override fun apply(proj: Project) {
    val extensions = proj.extensions
    val ext = extensions.create<WellgroundedExtensions>("wellgrounded")
    proj.task("wellgrounded") {
      doLast {
        repeat(ext.count) {
          println("Hello from Gradle")
        }
      }
    }
  }
}

apply<WellgroundedPlugin>()

configure<WellgroundedExtensions> {
  count = 4
}
```

All the power of our programming language is available within our plugins.

If you extract a plugin to another library, you can include it in your build through the same mechanism we saw earlier for including the SpotBugs plugin, as shown here:

```
plugins {
  id("com.wellgrounded.gradle") version "1000.0"
}

apply<WellgroundedPlugin>()

configure<WellgroundedExtensions> {
  count = 4
}
```

Summary

- Build tools are central to how Java software is constructed in the real world. They automate tedious operations, help with dependency management, ensure that developers are doing their work consistently, and, critically, ensure that the same project built on different machines gets the same results.
- Maven and Gradle are the two most common build tools in the Java ecosystem, and most tasks can be accomplished in either.
  - Maven takes an approach of configuration via XML combined with plugins written in JVM code.
  - Gradle provides a declarative build language using an actual programming language (Kotlin or Groovy), resulting in concise build logic for simple cases and flexibility for complex use cases.
- Dealing with conflicting dependencies is a major topic whatever your build tool. Both Maven and Gradle give you ways to handle conflicting library versions. Gradle provides a number of more advanced features for dealing with common dependency management issues.
- Gradle offers features for work avoidance such as incremental builds, resulting in faster builds.
- Modules, as seen in chapter 2, require some changes to our build scripting and source code layout, but these are well supported by the tooling.

Previous chapter
< Part 4. Build and deployment

Next chapter
12 Running Java in containers >